# COMP0222 Simultaneous Localisation and Mapping

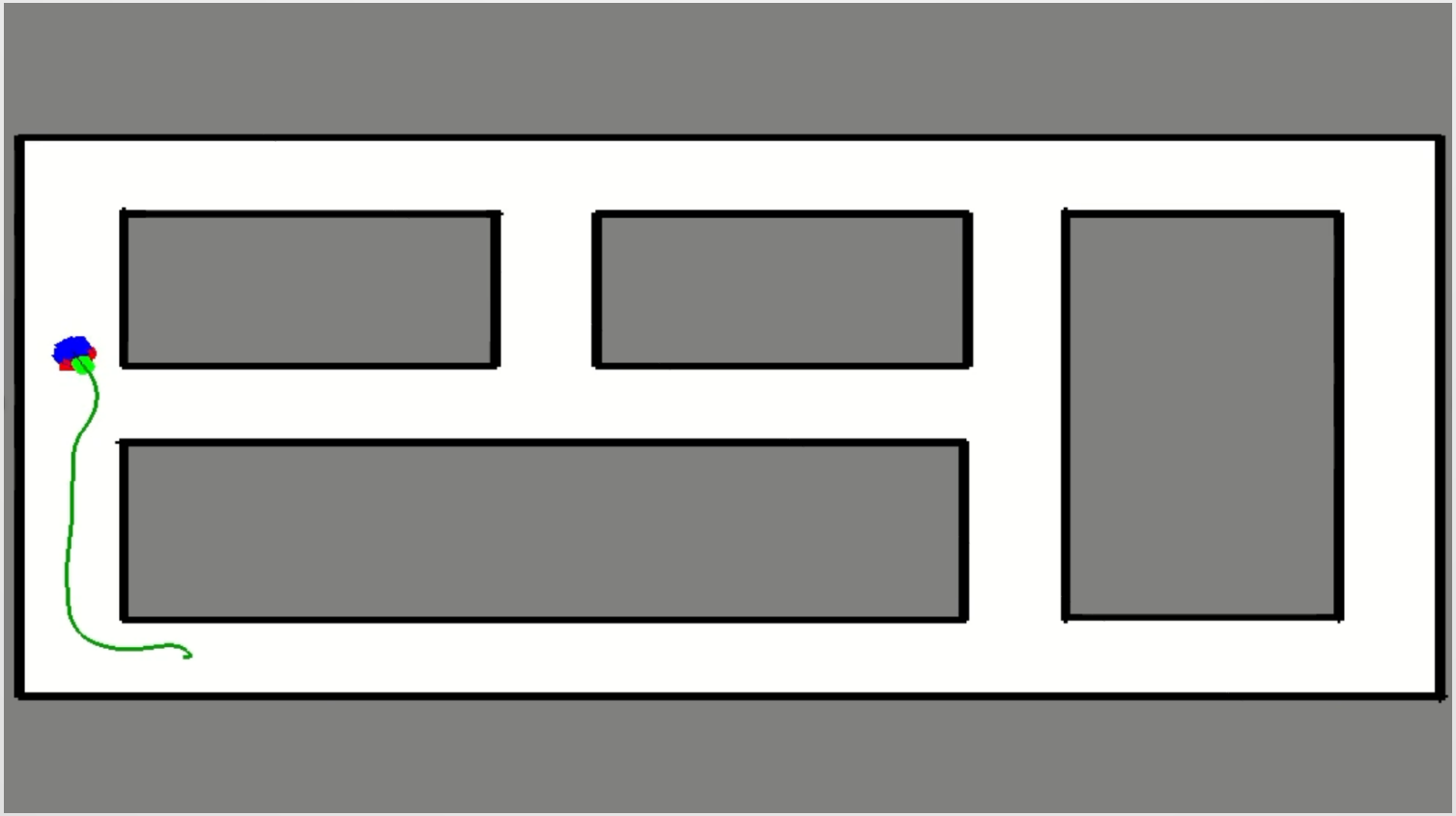
## Lecture 14: Sampling and Monte Carlo Localization

**Simon Julier**
s.julier@ucl.ac.uk
**Sajad Saeedi**
s.saeedi@ucl.ac.uk

## Limitations of Point-Like Estimates

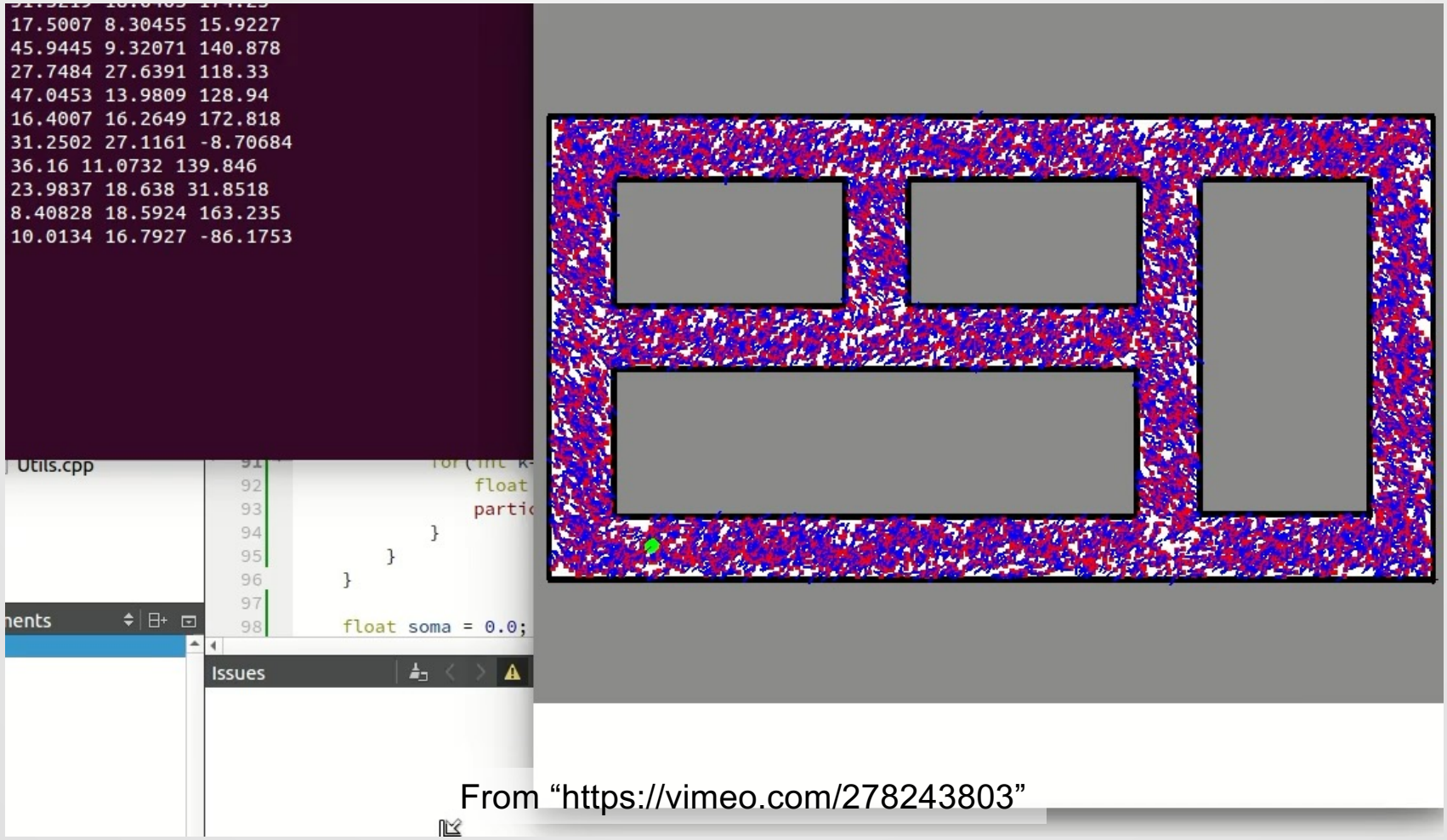
- So far, we've seen how to use Kalman filters and factor graphs to set up estimation problems
- We've seen how to extract point-like estimates from these estimators
- Both of these generate "point-like" estimates, with a measure of uncertainty
- Typically, we think of point-like estimates as being pretty "Gaussian-like"

# Gaussian-Like



# Limitations of Point-Like Estimates

- However, in some cases the distribution can be very non-Gaussian
- They can be multimodal and have multiple solutions




From “https://vimeo.com/278243803”

## Limitations of Point-Like Estimates

- However, in some cases the distribution can be very non-Gaussian
- They can be multimodal and have multiple solutions
- They can mix discrete and continuous random variables

## Bayesian Filtering and Particle Filters

- In principle, these problems can all be addressed using *Bayesian filtering* techniques
- However, these are often impractical
- Therefore, in this lecture, we'll look at sample-based approaches which approximate PDFs using discrete samples
- We'll see how these can be used to create a particle filter
- These are used to implement the 2D and 3D occupancy grids, which are the last techniques we'll review in this module

## Structure

- Problem Setup
- Particle Approximation
- Sampling Particles
- The Bootstrap Filter
- Effective Sample Size and Resampling
- A More Interesting Example

## Problem Setup

- **Problem Setup**
- ***Particle Approximation***
- ***Sampling Particles***
- ***The Bootstrap Filter***
- ***Effective Sample Size and Resampling***
- ***A More Interesting Example***

## Problem Setup

- In this lecture we mostly won't look at a SLAM context
- The process model is

$$\mathbf{x}_{k+1} = \mathbf{f}\left[\mathbf{x}_k, \mathbf{u}_{k+1}, \mathbf{v}_{k+1}\right]$$

- The observation model is

$$\mathbf{z}_{k+1} = \mathbf{h}\left[\mathbf{x}_{k+1}, \mathbf{u}_{k+1}, \mathbf{w}_{k+1}\right]$$

## Bayesian Filtering Problem

- **The goal is to estimate the *probability density function* of the state of interest,**

$$f\left(\mathbf{x}_k \middle| \mathbf{Z}_{0:k}, \mathbf{U}_{0:k}, \mathbf{x}_0\right)$$

- **Once we've got this, we can derive any quantity and precisely and correctly from our system**

## Bayesian Filtering Problem

- To simplify our equations, we'll introduce the notation

$$\mathbb{I}_k = \{\mathbf{Z}_{0:k}, \mathbf{U}_{0:k}, \mathbf{x}_0\}$$

- Therefore, we can rewrite the filtering problem as

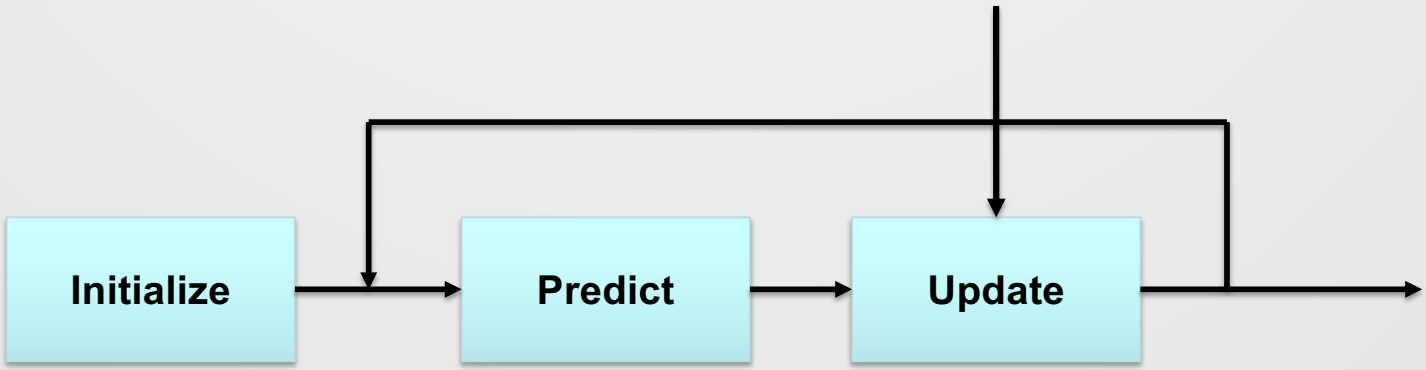
$$f\left(\mathbf{x}_k \middle| \mathbb{I}_k\right)$$

# Bayesian Filtering Problem and the PDF

- **For example, the mean is given by**

$$\hat{\mathbf{x}}_k = \int \mathbf{x}' f\left(\mathbf{x}' | \mathbb{I}_k\right) \mathrm{d}\mathbf{x}'$$

# Overview of the Bayesian Filter



## Initialization

- **The initial state is given by,**

$$f\left(\mathbf{x}_0\right)$$

## Prediction Step

- **The prediction is computed using the Chapman-Kolmogorov Equation**

$$f\left(\mathbf{x}_{k+1}|\mathbb{I}_k, \mathbf{u}_{k+1}\right) = \int f\left(\mathbf{x}_{k+1}|\mathbf{x}', \mathbf{u}_{k+1}\right) f\left(\mathbf{x}'|\mathbb{I}_k\right) \mathrm{d}\mathbf{x}'$$

## Measurement Update

- **The update is computed using Bayes' Rule,**

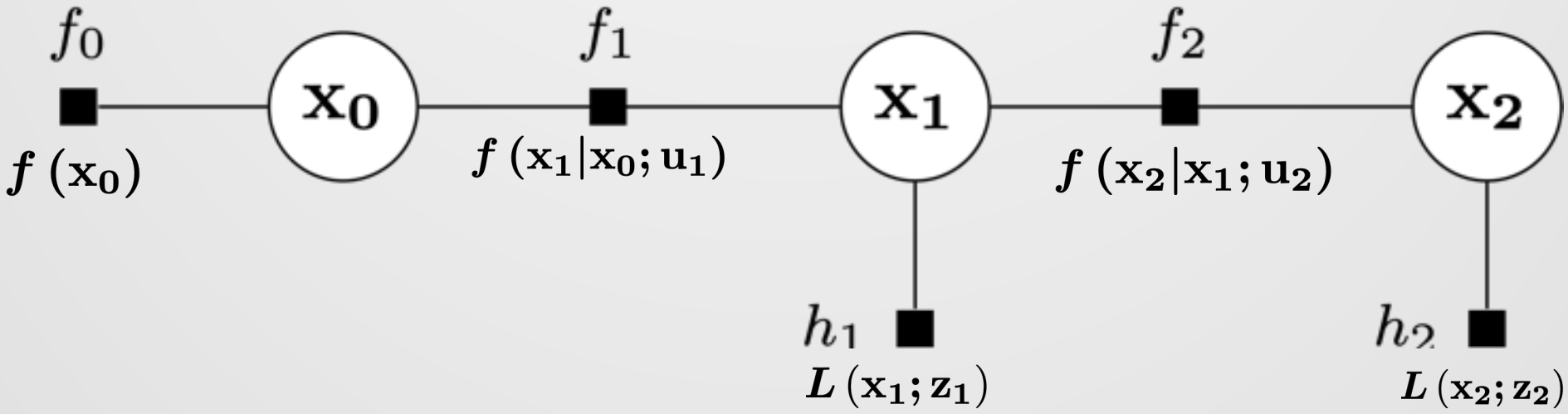
$$f\left(\mathbf{x}_{k+1}|\mathbb{I}_{k+1}\right) = \frac{f\left(\mathbf{z}_{k+1}|\mathbf{x}_{k+1}\right) f\left(\mathbf{x}_{k+1}|\mathbb{I}_{k}, \mathbf{u}_{k+1}\right)}{f\left(\mathbf{z}_{k+1}|\mathbb{I}_{k}, \mathbf{u}_{k+1}\right)}$$

$$f\left(\mathbf{z}_{k+1}|\mathbb{I}_{k}, \mathbf{u}_{k+1}\right) = \int f\left(\mathbf{z}_{k+1}|\mathbf{x}'\right) f\left(\mathbf{x}'|\mathbb{I}_{k}, \mathbf{u}_{k+1}\right) d\mathbf{x}'$$

# Limitations of the Bayesian Filter

- The PDF can be of arbitrary complexity
- To implement the filter, we have to compute a bunch of integrals we can't solve
- What about a factor graph? Didn't we say that contained all the information in a PDF?
- Why can't we use that?

# Factor Graphs



# Factor Graphs

$$\ln f\left(\mathbf{x}_{0:k}|\mathbf{z}_{1:k};\mathbf{u}_{1:k}\right) = -\frac{1}{2}\left(\mathbf{x}_0-\bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1}\left(\mathbf{x}_0-\bar{\mathbf{x}}_0\right)$$
$$-\frac{1}{2}\sum_{i=1}^{k}\mathbf{e}\left[\mathbf{x}_i,\mathbf{x}_{i-1},\mathbf{u}_i\right]^\top \mathbf{Q}_i^{-1}\mathbf{e}\left[\mathbf{x}_i,\mathbf{x}_{i-1},\mathbf{u}_i\right]$$
$$-\frac{1}{2}\sum_{i=1}^{k}\mathbf{l}\left[\mathbf{z}_i,\mathbf{x}_i\right]^\top \mathbf{R}_i^{-1}\mathbf{l}\left[\mathbf{z}_i,\mathbf{x}_i\right]$$

## Factor Graphs

- The factor graph gives us an expression for the *unnormalized* log joint probability over the time history of the state
- Therefore, it contains all the information we need
- However, we need to do real-time inference and MAP is about the best we can do
- We can try tricks such as initialize from lots of different locations to try to figure out the local minima
- However, this can be slow and unreliable

## Use Different Representations

- Rather than work directly with the equations, another approach is to try to address the problem *numerically*
- Rather than represent the probability as a function that we manipulate, represent it as a set of samples that we manipulate
- This leads to particle approximations and particle filters

# Particle Approximation

- *Problem Setup*
- Particle Approximation
- *Sampling*
- *The Bootstrap Filter*
- *Effective Sample Size and Resampling*
- *A More Interesting Example*

## Particle Approximation

- Suppose a random variable is distributed according to some probability density function,

$$\mathbf{x}_k \sim f\left(\mathbf{x}_k\right)$$

- We would like to get the shape of what the probability distribution looks like
- We can do this by drawing a lot of samples from the probability distribution

## Particle Approximation

- The idea is to approximate the continuous PDF by a set of $N$ discrete points or samples
- The $i$th point is written as

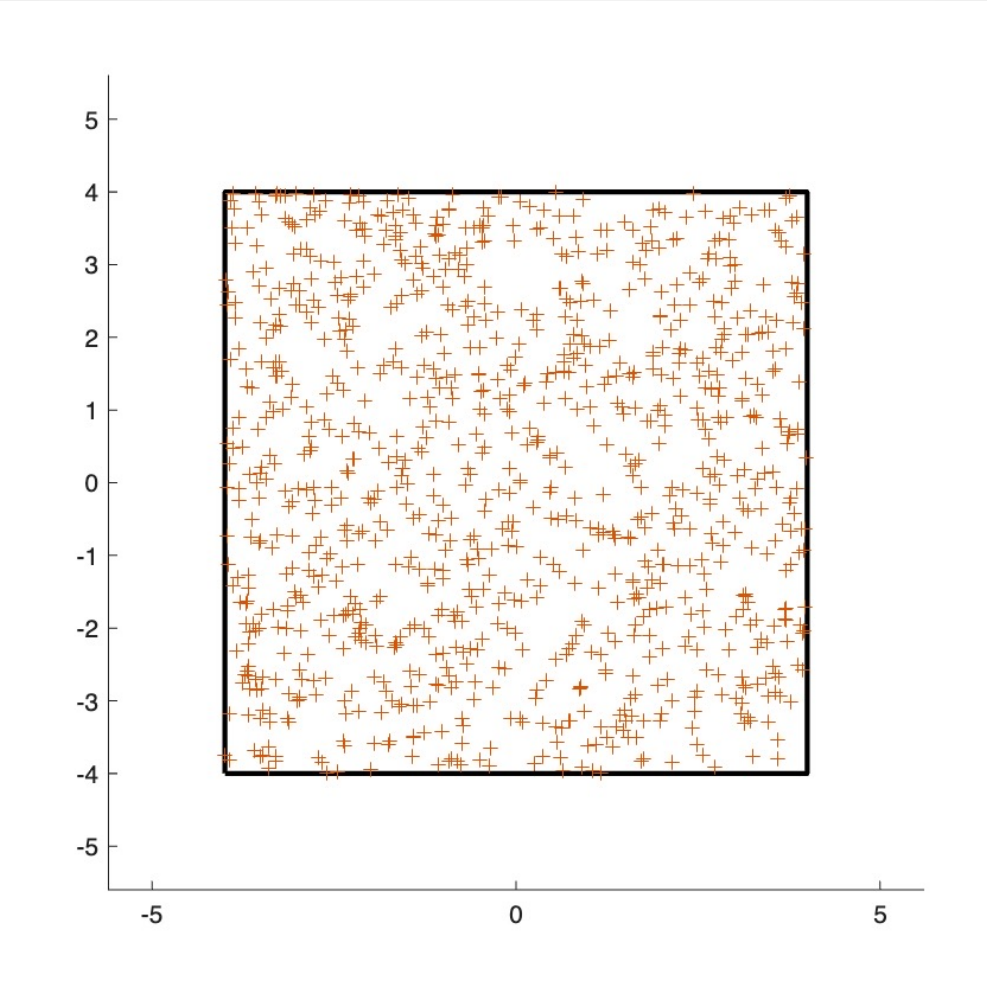
$$\mathbf{x}_k^{[i]}$$

## Particle Approximation

- **Suppose we have a 2 dimensional, uniformly distributed random variable in the square with sides of length $2L$,**

$$f(\mathrm{x}) = \begin{cases} \frac{1}{4L^2} & \text{for } -\frac{1}{L} \leq x, y \leq \frac{1}{L} \\ 0 & \text{otherwise} \end{cases}$$

- **The particle approximation is created by sampling $N$ points uniformly in the square**


5
4
3
2
1
0
-1
-2
-3
-4
-5
-5
0
5

## Particle Approximation

- **Technically the PDF represented by these discrete points is**

$$\hat{f}(\mathbf{x}) = \frac{1}{N}\sum_{i=1}^{N} \delta_{\mathbf{x}_k^{[i]}}(\mathbf{x})$$

- **The delta function has the value**

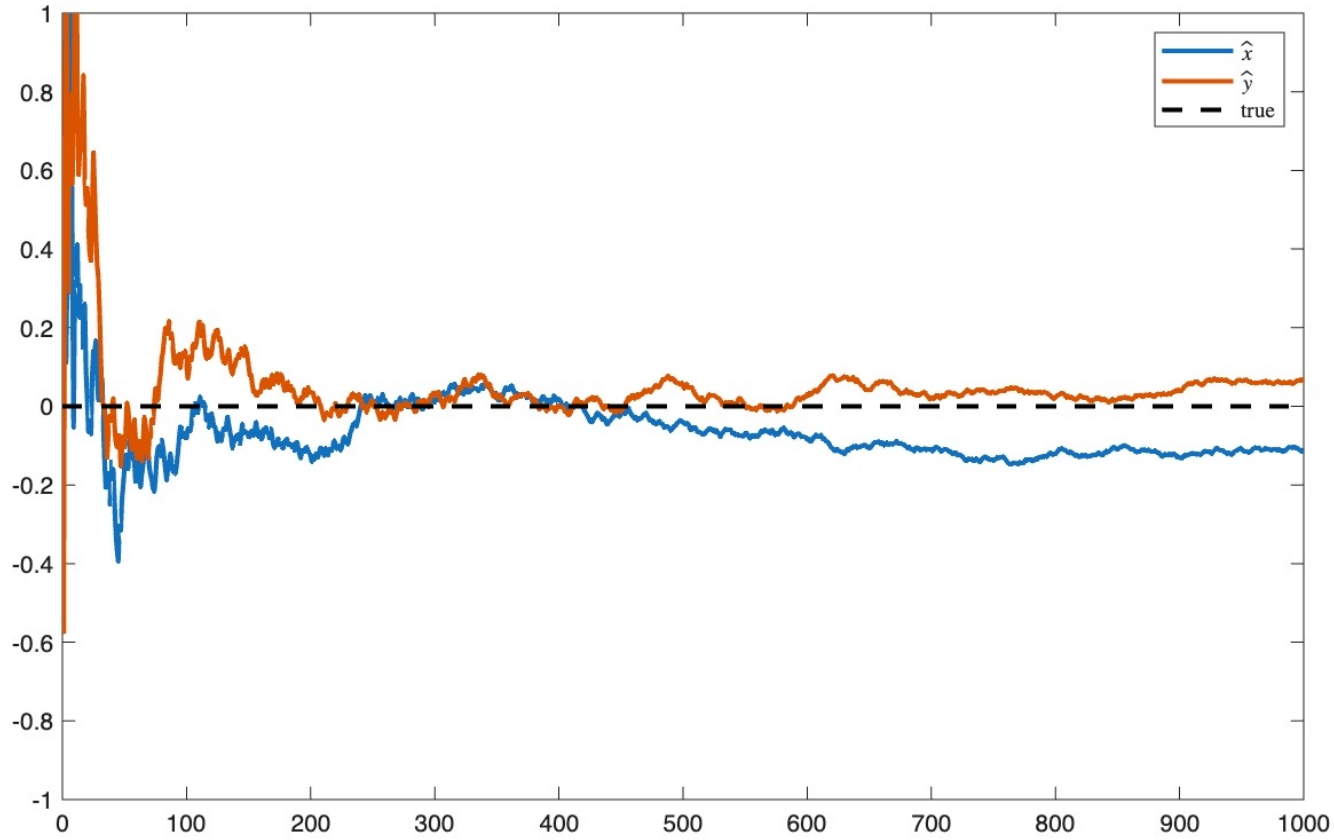
$$\delta_{\mathbf{x}_k^{[i]}}(\mathbf{x}) = \begin{cases} 1 & \text{if } \mathbf{x} = \mathbf{x}_k^{[i]} \\ 0 & \text{otherwise} \end{cases}$$

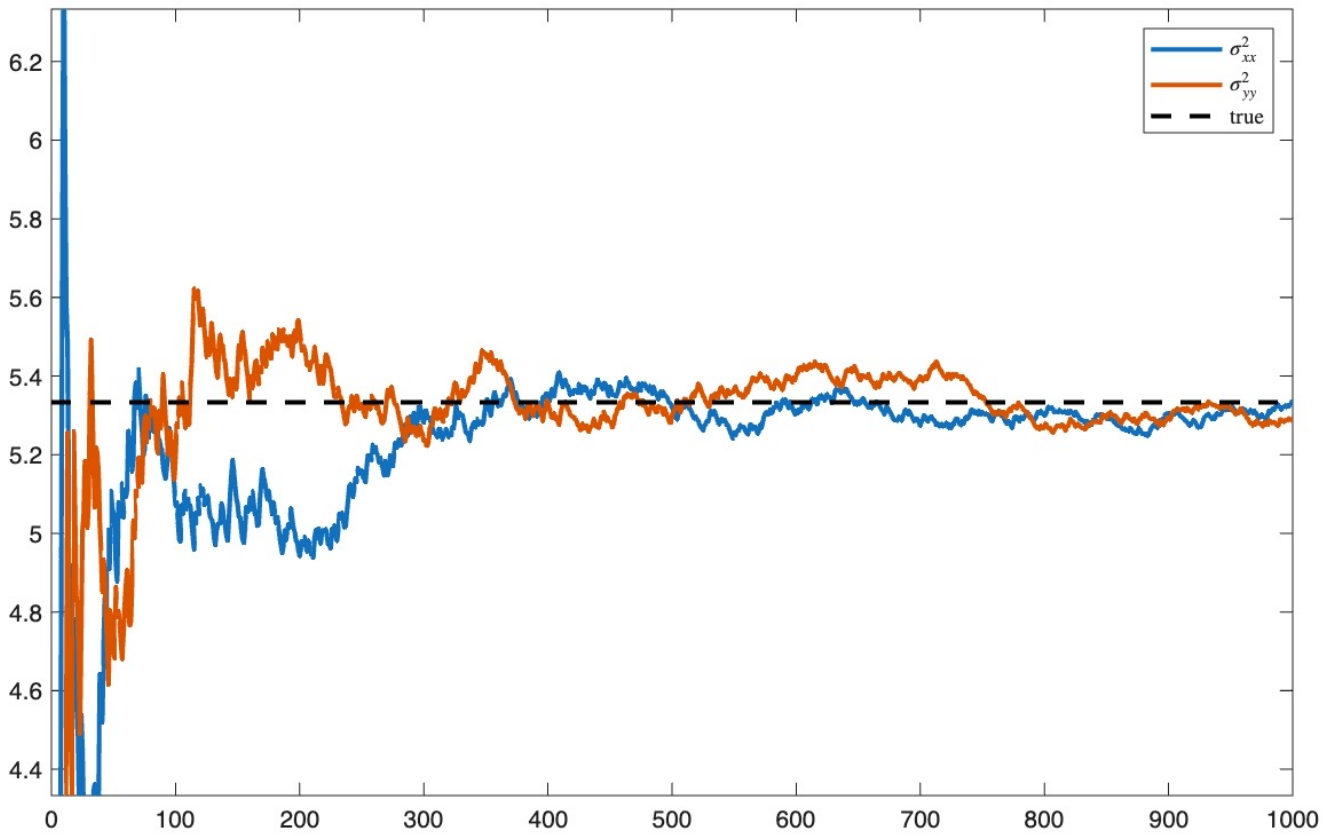
## Working with Particle Approximations

- We can use these to approximate integrals
- For example, the mean and covariance are given by

$$\hat{\mathbf{x}}_k \approx \frac{1}{N} \sum_{i=1}^{N} \mathbf{x}_k^{[i]}$$

$$\mathbf{P}_k \approx \frac{1}{N} \sum_{i=1}^{N} \left( \mathbf{x}_k^{[i]} - \hat{\mathbf{x}} \right)^\top \left( \mathbf{x}_k^{[i]} - \hat{\mathbf{x}} \right)$$


1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
100
200
300
400
500
600
700
800
900
1000
$\hat{x}$
$\hat{y}$
true


6.2
6
5.8
5.6
5.4
5.2
5
4.8
4.6
4.4
0
100
200
300
400
500
600
700
800
900
1000
$\sigma^2_{xx}$
$\sigma^2_{yy}$
true

## Where do the Particles Come From?

- We can see that doing things like computing moments simply becomes a matter of adding and dividing functions
- However, where do the samples  come from?
- One way to do this is through *sampling algorithms*

# Sampling

- *Problem Setup*
- *Particle Approximation*
- Sampling
- *The Bootstrap Filter*
- *Effective Sample Size and Resampling*
- *A More Interesting Example*

## The Sampling Problem

- Suppose we have a PDF

$$\mathbf{x}_k \sim f\left(\mathbf{x}_k\right)$$

- Sampling is the process by which we draw a bunch of samples,

$$\mathbf{x}_k^{[1]},\ \mathbf{x}_k^{[2]},\ \ldots,\ \mathbf{x}_k^{[N]}$$

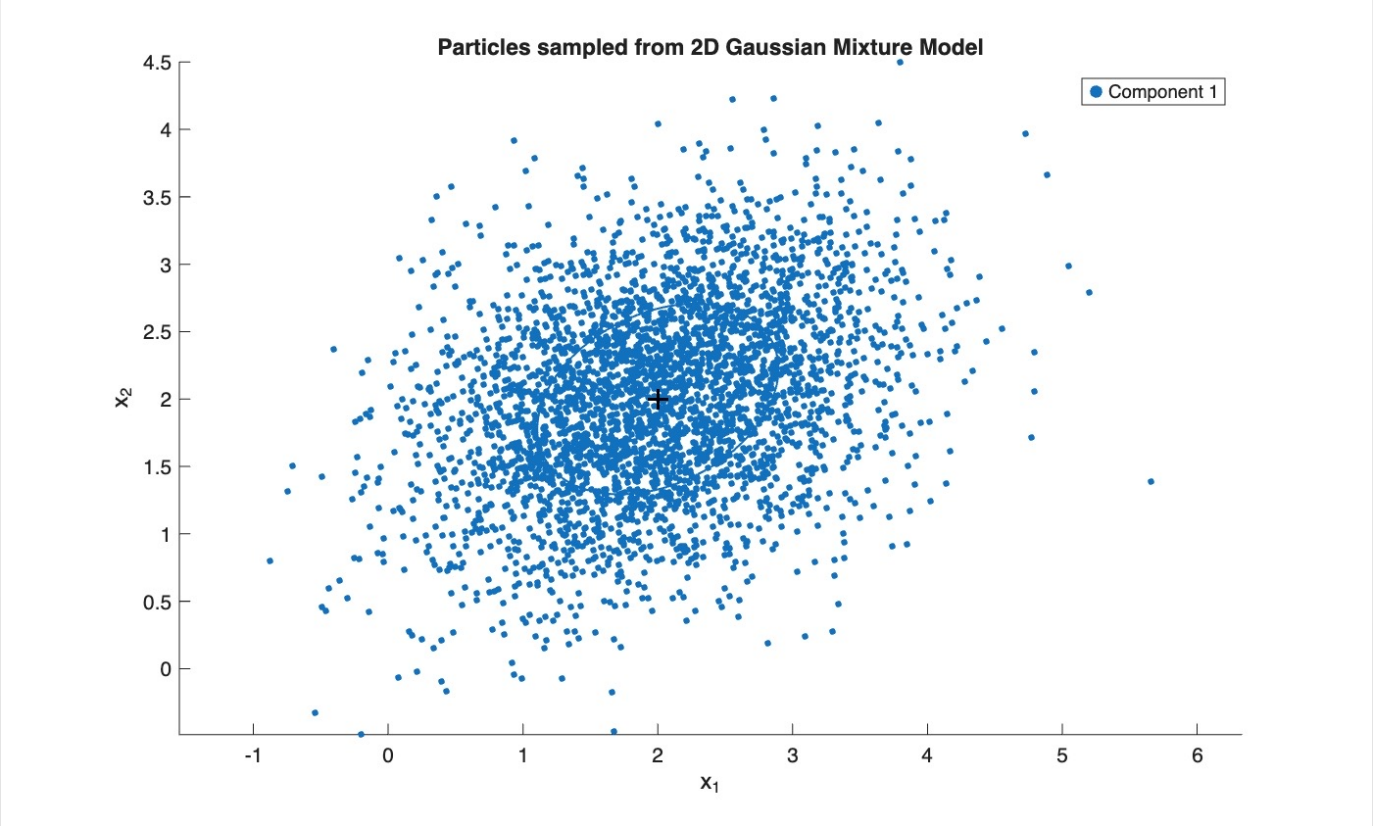
- The statistical properties of these samples have to be the same, in the limit, as the properties of $f()$

## Sampling

- For some PDFs we can directly generate samples
- We've already seen it for uniform PDFs
- We can also easily do it for Gaussians,

$$f(\mathrm{x}) = \mathcal{G}(\mathrm{x}; \mu, \Sigma)$$

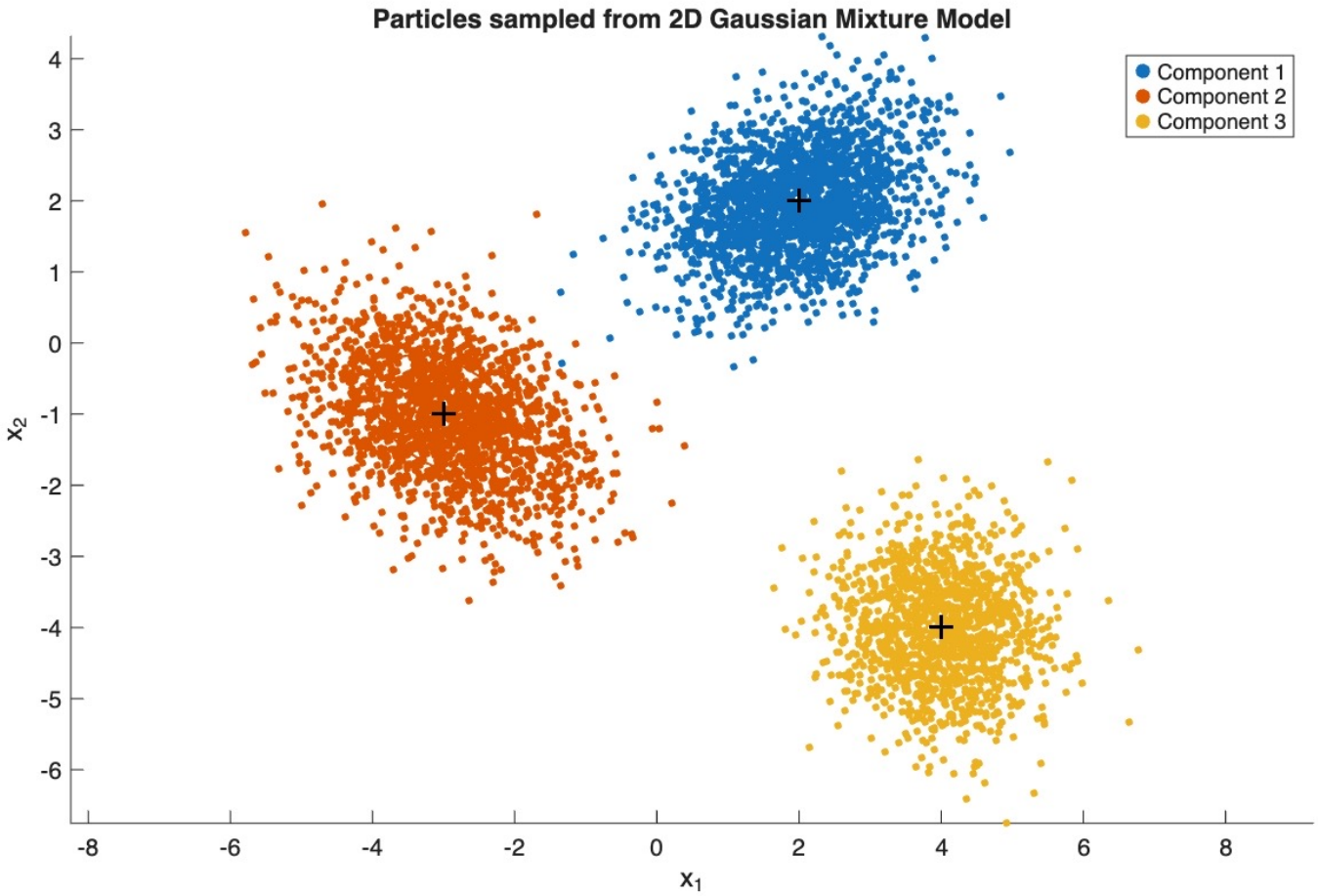
# Sample from a Gaussian

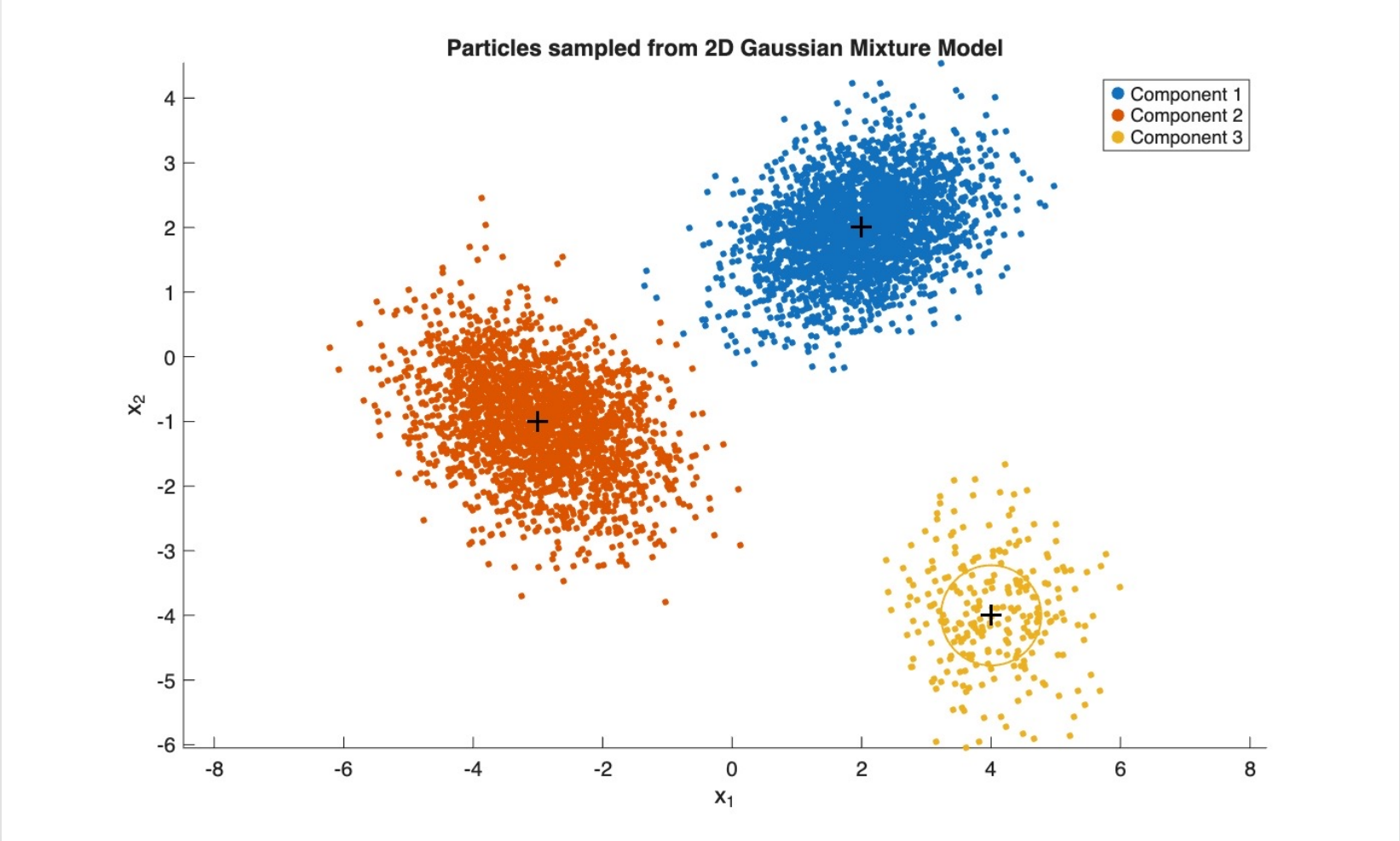


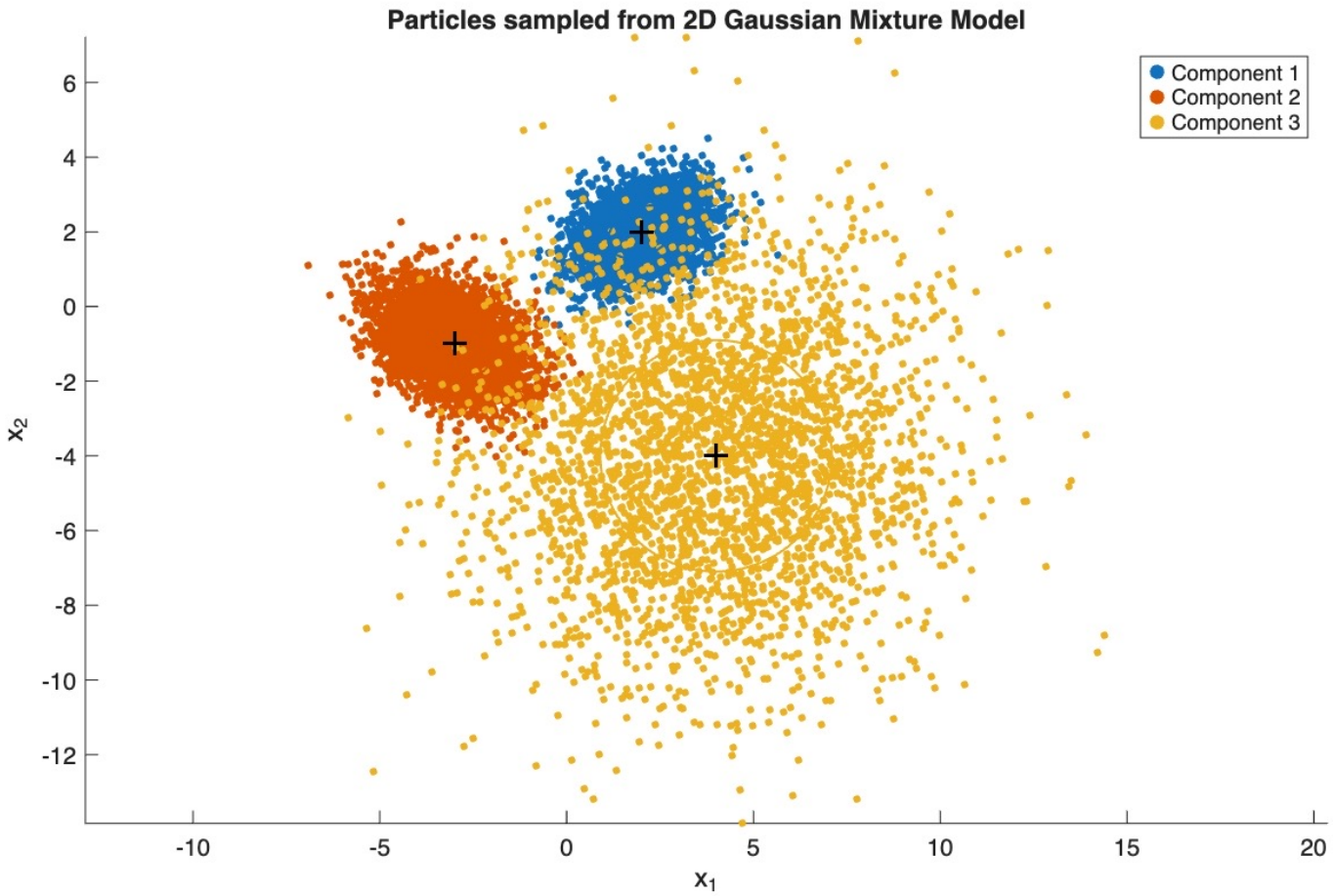
## Particle Approximation of a Gaussian Mixture Model

- **We can extend this to a mixture of Gaussians such as**

$$f(\mathrm{x}) = w_1 \mathcal{G}(\mathrm{x}; \mu_1, \Sigma_1) + w_2 \mathcal{G}(\mathrm{x}; \mu_2, \Sigma_2) + w_3 \mathcal{G}(\mathrm{x}; \mu_3, \Sigma_3)$$


Particles sampled from 2D Gaussian Mixture Model
Component 1
Component 2
Component 3
4
3
2
1
0
-1
-2
-3
-4
-5
-6
$x_2$
-8
-6
-4
-2
0
2
4
6
8
$x_1$


Particles sampled from 2D Gaussian Mixture Model
Component 1
Component 2
Component 3
$x_1$
$x_2$
-8
-6
-4
-2
0
2
4
6
8
4
3
2
1
0
-1
-2
-3
-4
-5
-6


Particles sampled from 2D Gaussian Mixture Model
Component 1
Component 2
Component 3
6
4
2
0
-2
-4
-6
-8
-10
-12
$x_2$
-10
-5
0
5
10
15
20
$x_1$

## Sampling and Particle Approximations

- However, in most cases we're given a PDF but we can't sample directly from it
- An example is the factor graph

# Factor Graphs

$$\ln f\left(\mathbf{x}_{0:k}|\mathbf{z}_{1:k};\mathbf{u}_{1:k}\right) = -\frac{1}{2}\left(\mathbf{x}_0-\bar{\mathbf{x}}_0\right)^\top \mathbf{P}_0^{-1}\left(\mathbf{x}_0-\bar{\mathbf{x}}_0\right)$$
$$-\frac{1}{2}\sum_{i=1}^{k}\mathbf{e}\left[\mathbf{x}_i,\mathbf{x}_{i-1},\mathbf{u}_i\right]^\top \mathbf{Q}_i^{-1}\mathbf{e}\left[\mathbf{x}_i,\mathbf{x}_{i-1},\mathbf{u}_i\right]$$
$$-\frac{1}{2}\sum_{i=1}^{k}\mathbf{l}\left[\mathbf{z}_i,\mathbf{x}_i\right]^\top \mathbf{R}_i^{-1}\mathbf{l}\left[\mathbf{z}_i,\mathbf{x}_i\right]$$

## Sampling

- We can handle these sorts of PDFs using *trajectory-based sampling*
- The sampling algorithm traces a trajectory through state space
- The shape of the trajectory is governed by the PDF
- Periodically, we grab the value of the trajectory and call that a sample
- A key approach for this is *rejection-based sampling*

# Rejection-Based Sampling

**Requires:** Unnormalized target density $\tilde{\pi}(x)$, proposal distribution $q(x' \mid x)$, initial state $x^{[0]}$, number of samples $N$

**Ensures:** Samples $\{x^{[i]}\}_{i=1}^{N}$

$x \leftarrow x^{[0]}$
**for** $i = 1$ to $N$ **do**
  Draw proposal $x' \sim q(x' \mid x)$
  $\alpha \leftarrow \min\left(1, \frac{\tilde{\pi}(x')\, q(x \mid x')}{\tilde{\pi}(x)\, q(x' \mid x)}\right)$
  Draw $u \sim \mathrm{Uniform}(0, 1)$
  **if** $u \leq \alpha$ **then**
    $x \leftarrow x'$
  **end if**
  $x^{[i]} \leftarrow x$
**end for**

## Draw Proposal Step

- In the draw proposal step, we suggests a new sample location according to the distribution

$$x' \sim q(x' \mid x)$$

- You can think of it like a “process model” for the sampling trajectory

## Draw Proposal Step

- **For example, if the proposal distribution were Gaussian,**

$$\text{Draw } \varepsilon \sim \mathcal{N}(0, \sigma^2 I)$$
$$x' \leftarrow x + \varepsilon$$

## Sample Acceptance Probability

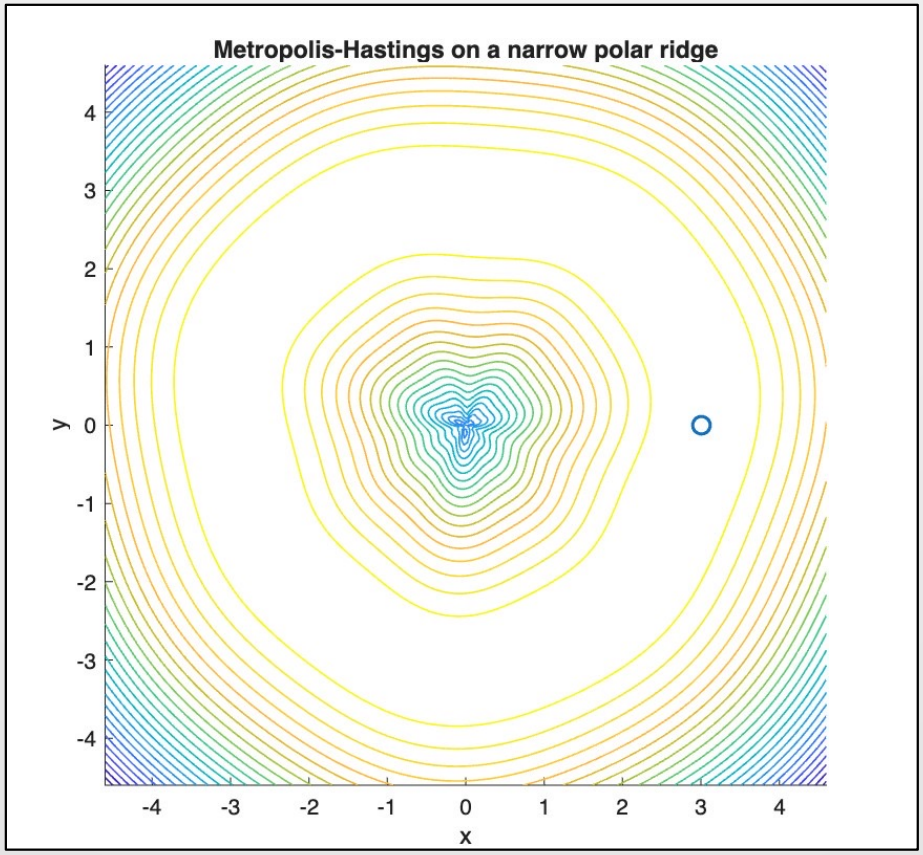
- **The probability of accepting the new sample is given by**

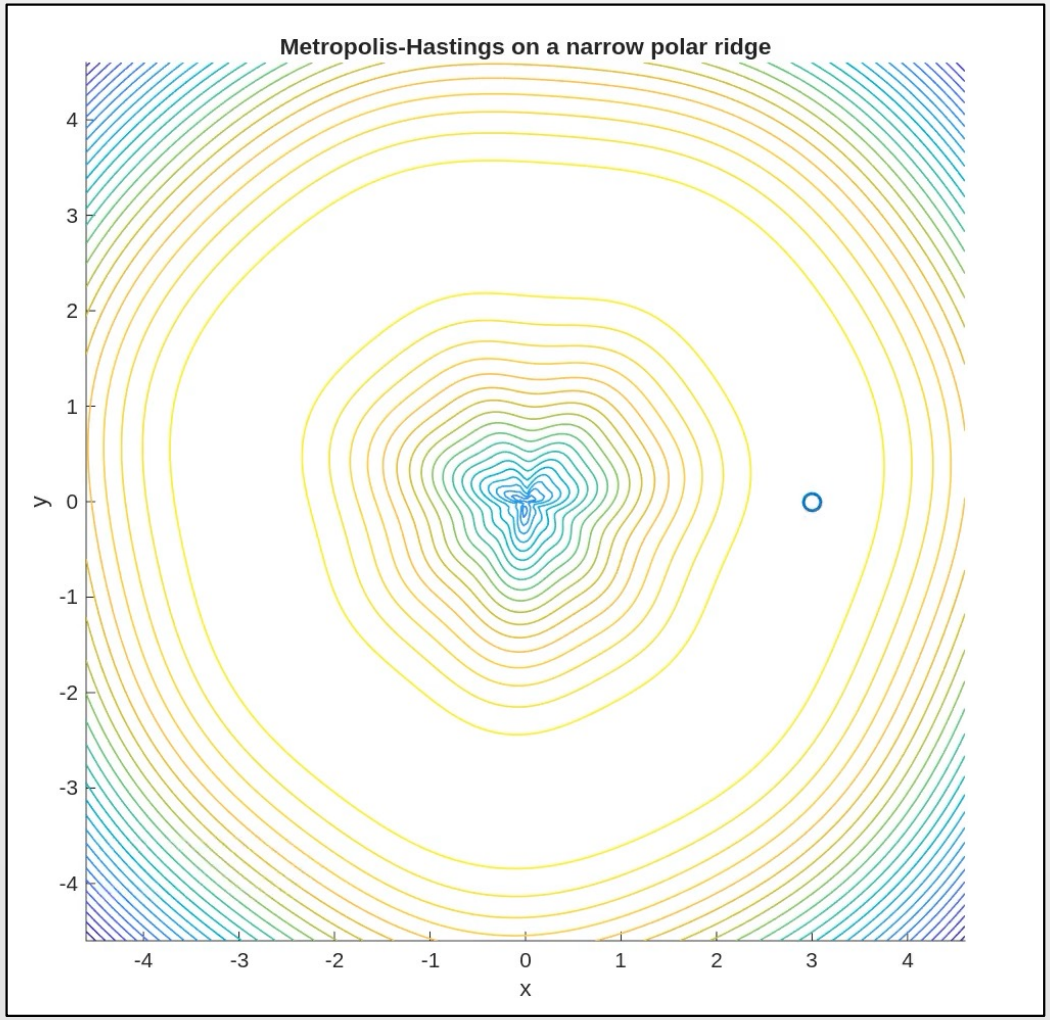
$$\alpha = \min\left(1, \frac{\tilde{\pi}(x')\, q(x \mid x')}{\tilde{\pi}(x)\, q(x' \mid x)}\right)$$

## Sample Acceptance Probability

$$\min\left(1, \frac{\tilde{\pi}(x')\, q(x \mid x')}{\tilde{\pi}(x)\, q(x' \mid x)}\right)$$

# Example


Metropolis-Hastings on a narrow polar ridge
4
3
2
1
0
-1
-2
-3
-4
y
-4
-3
-2
-1
0
1
2
3
4
x


Metropolis-Hastings on a narrow polar ridge
y
x
4
3
2
1
0
-1
-2
-3
-4
-4
-3
-2
-1
0
1
2
3
4

## Sampling and Factor Graphs

- Given a PDF, we can use sampling algorithms to generate particles which we can use to do further inferences
- However, we want to do *real-time* operation for our robots
- Therefore, this approach is too slow and awkward
- However, it turns out there is a way to directly use the particles to build a particle filter algorithm
- We will see how to develop the bootstrap filter, which is a simple particle filter

# The Bootstrap Particle Filter

- *Problem Setup*
- *Particle Approximation*
- *Sampling*
- **The Bootstrap Particle Filter**
- *Effective Sample Size and Resampling*
- *A More Interesting Example*

## Bootstrap Particle Filter

- The bootstrap filter is one of the oldest particle filters which has been developed
- There are many modifications which have been made to improve its performance
- However, it is simple to explain and demonstrates many of the key issues

## Weighted Particle Approximations

- **The particle filter represents its estimated using a *weighted* set of particles,**

$$\left\{ \left( w_{j|k}^{[i]}, \mathbf{x}_{j|k}^{[i]} \right) : i = 1, \cdots, N \right\}$$

$$w_{j|k}^{[i]} \geq 0 \qquad \sum_{i=1}^{N} w_{j|k}^{[i]} = 1$$

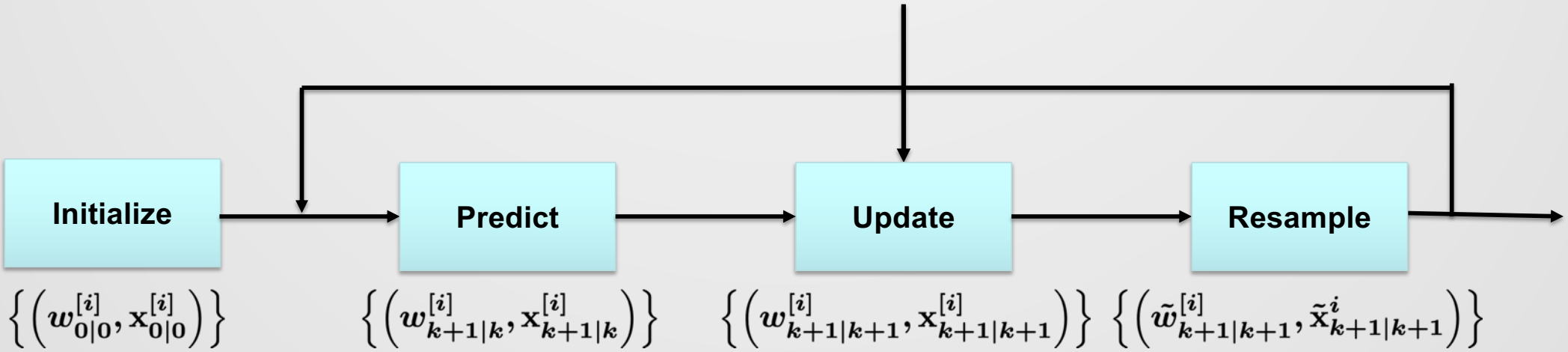
## Weighted Particle Approximations

- The weights are used in any operation
- For example, the mean and covariance are computed from

$$\hat{\mathbf{x}}_{j|k} \approx \frac{1}{N} \sum_{i=1}^{N} w_{j|k}^{[i]} \mathbf{x}_{j|k}^{[i]}$$

$$\mathbf{P}_{j|k} \approx \frac{1}{N} \sum_{i=1}^{N} w_{j|k}^{[i]} \left( \mathbf{x}_{j|k}^{[i]} - \hat{\mathbf{x}}_{j|k} \right)^{\top} \left( \mathbf{x}_{j|k}^{[i]} - \hat{\mathbf{x}}_{j|k} \right)$$

# Overview of the Bootstrap Filter



## Linear Example

- We'll start with a linear example which totally fails to exploit any of the generality of a particle filter
- The state is position and velocity in 2D

$$\mathbf{x}_k = \begin{bmatrix} x_k & \dot{x}_k & y_k & \dot{y}_k \end{bmatrix}^\top$$

- Motion is piecewise constant velocity
- The observation vector is

$$\mathbf{z}_k = \begin{bmatrix} x_k & y_k \end{bmatrix}^\top$$

Figure 1
File Edit View Insert Tools Desktop Window Help
40
30
20
10
0
-10
-20
-30
-40
-80
-60
-40
-20
0
20
40
60
17:20
23/02/2021

## Initialize

- In the initialization step we just create an in initial population of particles with the same weight
- For example the prior might be a Gaussian and so

$$\mathbf{x}_{0|0}^{[i]} \sim \mathcal{G}\left(\mathbf{x}; \bar{\mathbf{x}}_0, \mathbf{P}_0\right)$$

- The weights are all set to the same value,

$$w_{0|0}^{[i]} = \frac{1}{N}$$

## Predict Particles

- The prediction step moves the particles to a new location
- The movement of the particles is governed by a proposal distribution,

$$\mathbf{x}' \sim q\left(\mathbf{x}' \mid \mathbf{x}_{j|k}^{[i]}\right)$$

## Predict Particles

- **In the bootstrap filter, the proposal distribution is the same as the state transition probability**

$$q\left(\mathbf{x}' \mid \mathbf{x}_{k|k}^{[i]}\right) = f\left(\mathbf{x}' | \mathbf{x}_{k|k}^{[i]}, \mathbf{u}_{k+1}\right)$$

## Predict Particles

- This is easy to implement: simply run each particle through the process model, sampling the process noise differently each time,

$$\mathbf{x}_{k+1|k}^{[i]} = \mathbf{f}\left[\mathbf{x}_{k|k}^{[i]}, \mathbf{u}_{k+1}, \mathbf{v}_{k+1}^{[i]}\right]$$

# Predict Particles

$$\mathbf{x}_{k+1|k}^{[i]} = \mathbf{f}\left[\mathbf{x}_{k|k}^{[i]}, \mathbf{u}_{k+1}, \mathbf{v}_{k+1}\right]$$

## Predict Weights

- **Although the particle positions change, the particle weights do *not***
- **Therefore,**

$$w_{k+1|k}^{[i]} = w_{k|k}^{[i]}$$

## Update

- **The update step applies Bayes Rule to each particle,**

$$p\left(\mathbf{x}_{k+1|k+1}^{[i]} | \mathbf{z}_{k+1}\right) = \frac{p\left(\mathbf{z}_{k+1} \mid \mathbf{x}_{k+1|k}^{[i]}\right) p\left(\mathbf{x}_{k+1|k}^{[i]}\right)}{p\left(\mathbf{z}_{k+1}\right)}$$

- **We can rewrite this directly in terms of the particle weights**

## Update and Particle Weights

- The particle weights are updated according to

$$w_{k+1|k+1}^{[i]} = \frac{p\left(\mathbf{z}_{k+1} \mid \mathbf{x}_{k+1|k}^{[i]}\right)}{p\left(\mathbf{z}_{k+1}\right)} \cdot w_{k+1|k}^{[i]}$$

- You can treat this as an observation-dependent scale factor which relates how closely the particle matches with the observation

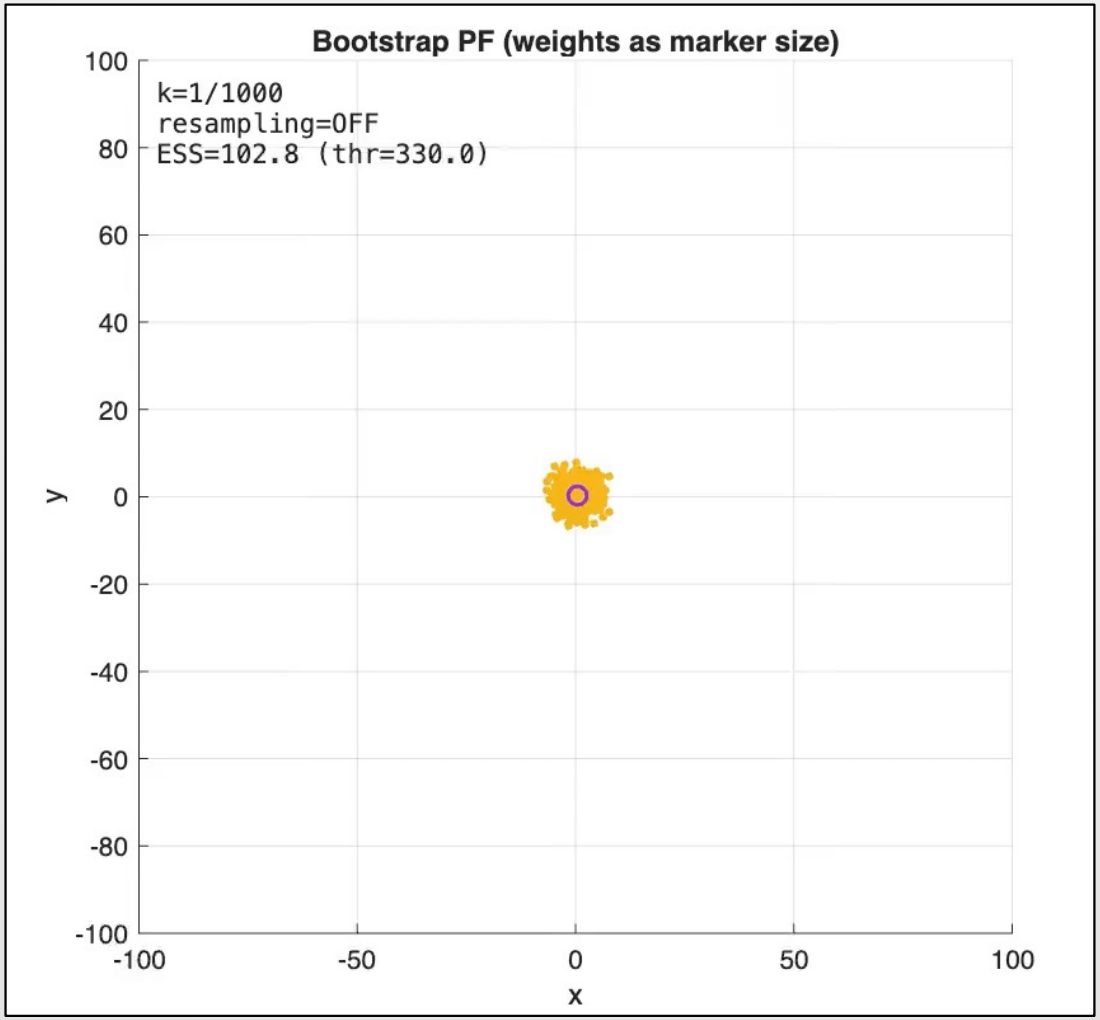
## Update and Particle Weights

- **The value at the bottom is normalizing constant,**

$$p\left(\mathbf{z}_{k+1}\right) = \sum_{i=1}^{N} p\left(\mathbf{z}_{k+1} \mid \mathbf{x}_{k+1|k}^{[i]}\right) w_{k+1|k}^{[i]}$$

## Okay... So Let's See it in Action

- We'll now show the bootstrap filter working on the linear system
- We'll draw the true trajectory (orange), the current estimate (red circle) and the particle locations
- The particle sizes are scaled to give an indication of the magnitude of the weight


Bootstrap PF (weights as marker size)
k=1/1000
resampling=OFF
ESS=102.8 (thr=330.0)
100
80
60
40
20
0
-20
-40
-60
-80
-100
y
-100
-50
0
50
100
x

## What Happened?

- The weights on almost all the particles went to zero
- The only particle with a non-zero weight *is the one closest to the most recent observation*
- This is called *particle depletion*
- It is caused by the repeated application of the update equation

## What Happened?

- Recall the weights are updated according to

$$w_{k+1|k+1}^{[i]} = \frac{p\left(\mathbf{z}_{k+1} \mid \mathbf{x}_{k+1|k}^{[i]}\right)}{p\left(\mathbf{z}_{k+1}\right)} \cdot w_{k+1|k}^{[i]}$$

- Particles far from the measurement lose their weight very quickly
- The division operation in the update assigns the particle with the highest weight a value of (nearly) 1

## Fixing Particle Depletion

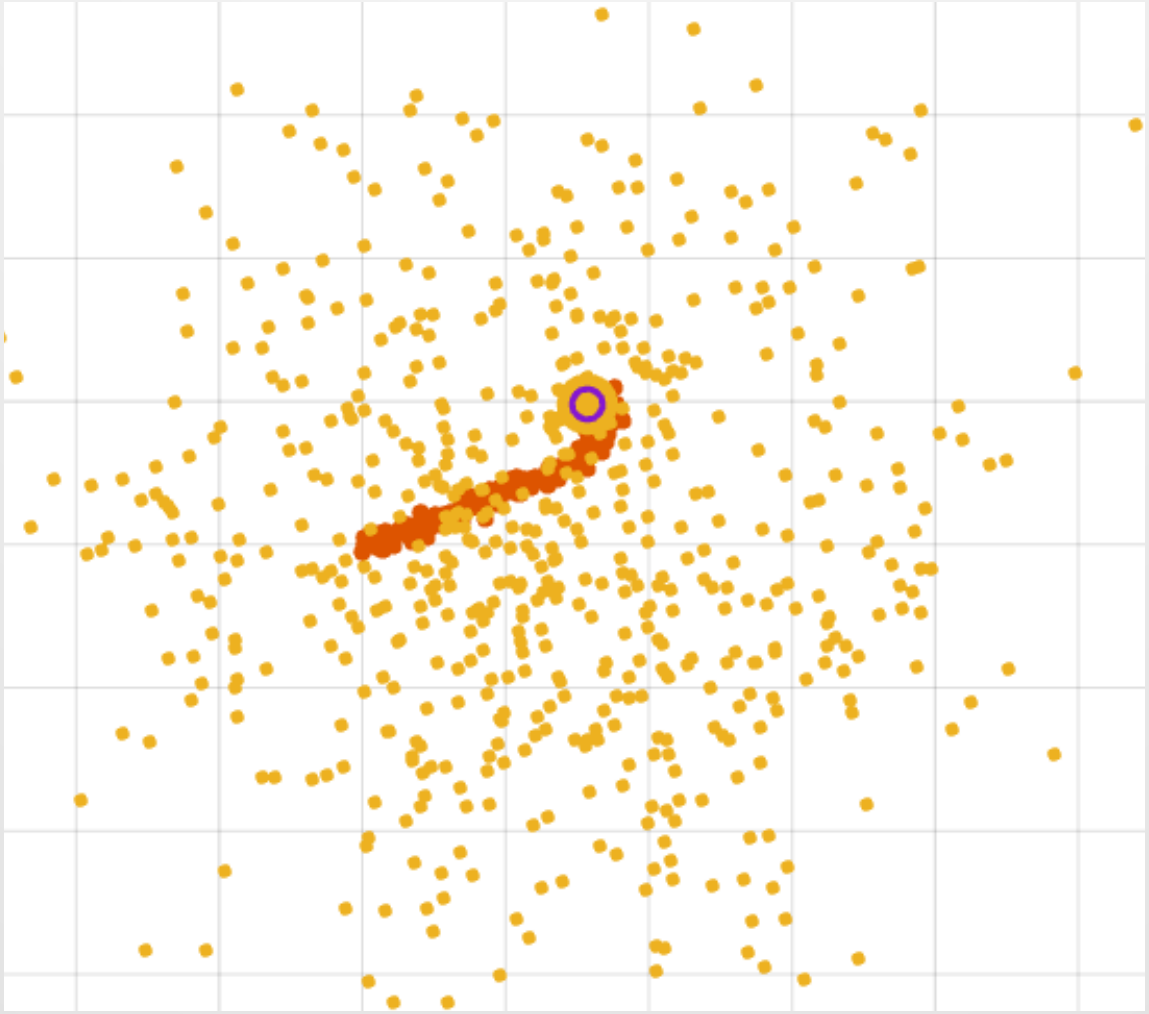
- This problem arises because the update equation *only* changes the weights on the particles
- It cannot change their positions to move them back closer to the observation
- As a result, as particles drift further from the observation, all we can do is set their weight to 0
- The mechanism we use to move the particles is called resampling

# Effective Sample Size and Resampling

- *Problem Setup*
- *Particle Approximation*
- *Sampling*
- *Bootstrap Filter*
- **Effective Sample Size and Resampling**
- *A More Interesting Example*

## Resampling

- Resampling changes the population of the particles so that they tend to be closer to the measurements
- Each particle has a probability of survival which is related to its weight
- We do sampling with replacement
  - This means we can choose the same sample over and over again
  - As a result, the total number of particles does not change



## When to Resample?

- We could resample every time step
- However, this can slow everything down
- Therefore, the normal way to do it is resample only when the particles become sufficiently depleted
- A way to measure depletion is the effective sample size (ESS)
- The ESS measures how many particles have a meaningful contribution to the estimate

# Effective Sample Size

- **The ESS is computed from,**

$$E_{k|k} = \frac{1}{\sum_{i=1}^{N} \left( w_{k|k}^{[i]} \right)^2}$$

## Effective Sample Size

- **First consider the case when all the particles have the *same weight*,**

$$w_{k|k}^{[i]} = \frac{1}{N}$$

- **The ESS is**

$$E_{k|k} = \frac{1}{\sum_{i=1}^{N} \left(\frac{1}{N}\right)^2}$$

## Effective Sample Size

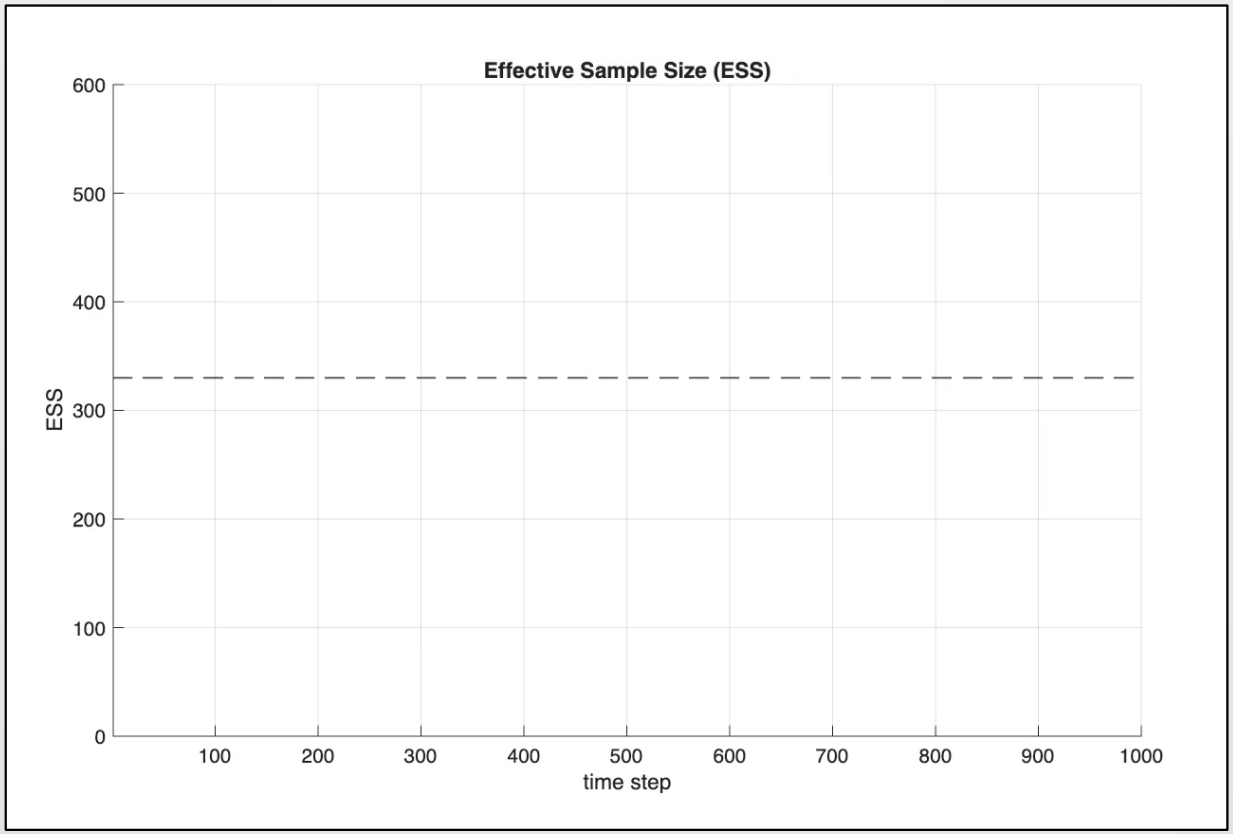
- Now consider the case when just one particle has a non-zero weight,

$$w_{k|k}^{[i]} = \begin{cases} 1 & \text{if } i = 1 \\ 0 & \text{otherwise} \end{cases}$$

- The ESS is

$$E_{k|k} = \frac{1}{\sum_{i=1}^{N} \left(\frac{1}{N}\right)^2}$$

# ESS for the Example

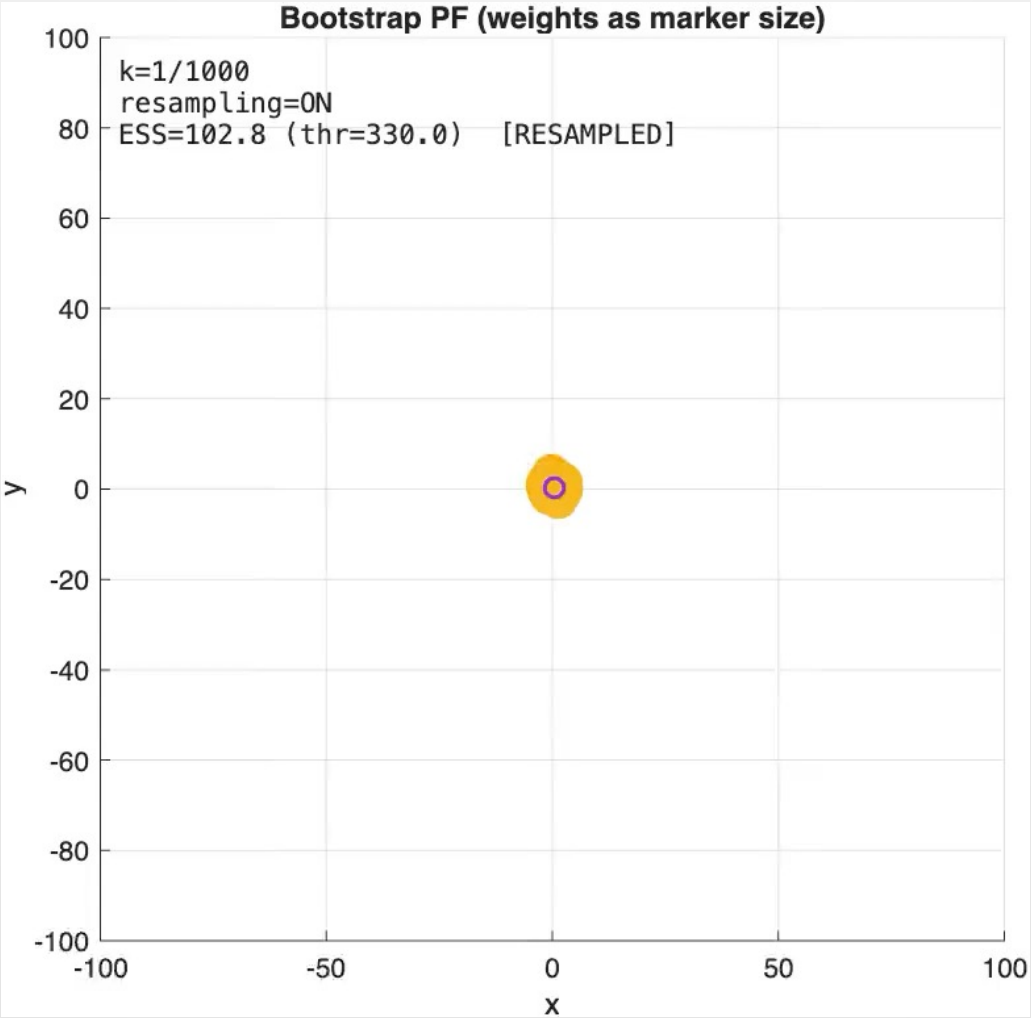


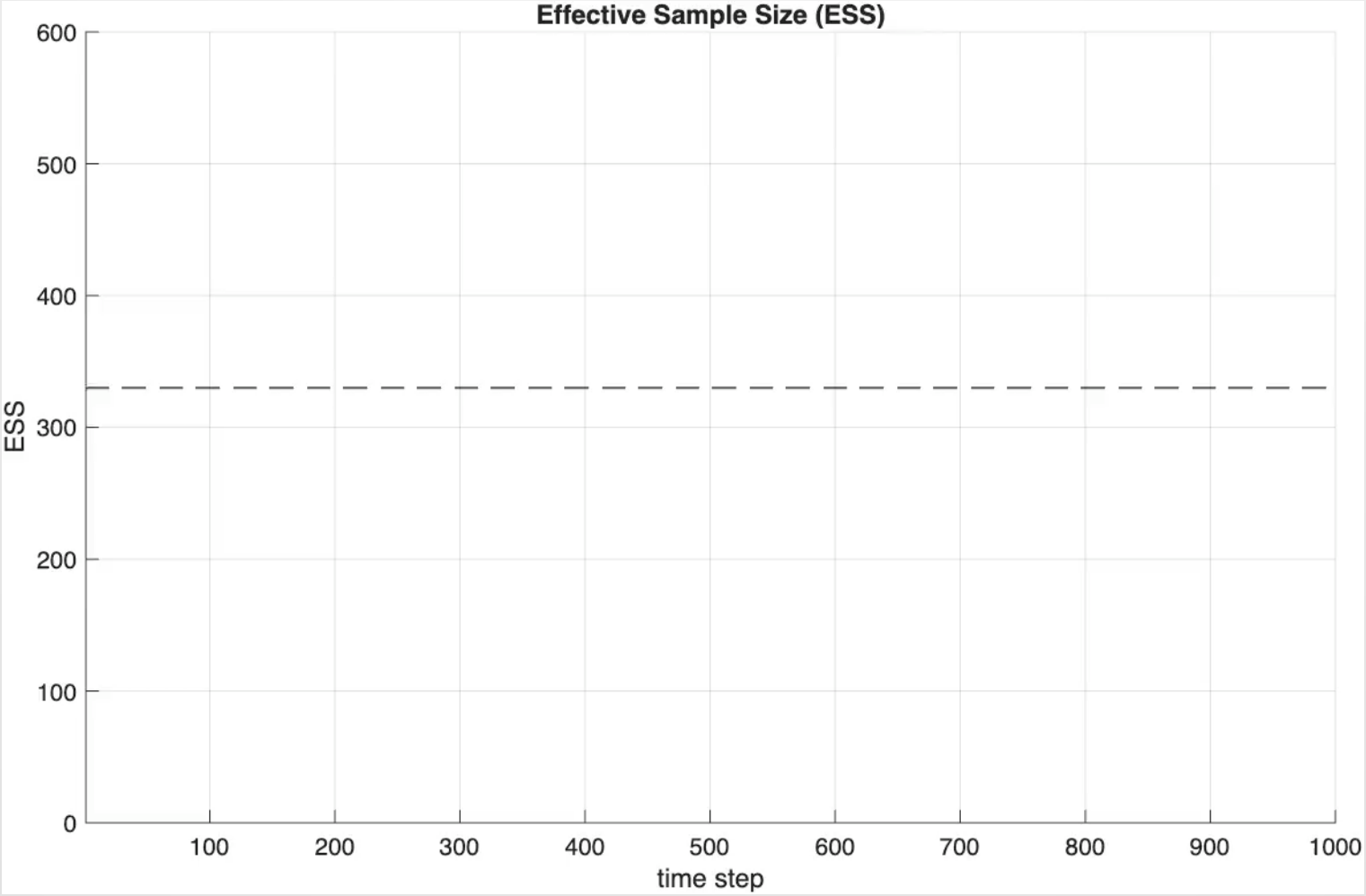
## Using the ESS

- The threshold for triggering a resample is when the ESS falls below some fraction of the total number of particles,

$$E_{k|k} < \alpha N$$

- Typically $\alpha = 0.6$


Bootstrap PF (weights as marker size)
k=1/1000
resampling=ON
ESS=102.8 (thr=330.0) [RESAMPLED]
100
80
60
40
20
0
-20
-40
-60
-80
-100
y
-100
-50
0
50
100
x

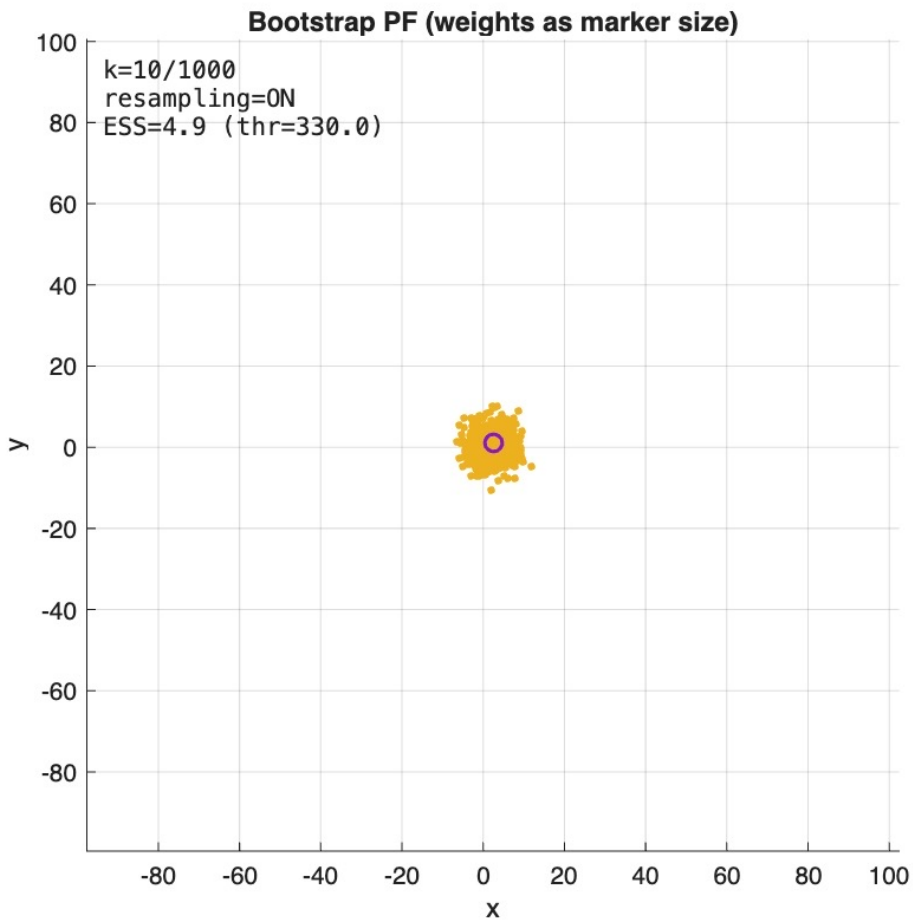

Effective Sample Size (ESS)
ESS
600
500
400
300
200
100
0
100
200
300
400
500
600
700
800
900
1000
time step

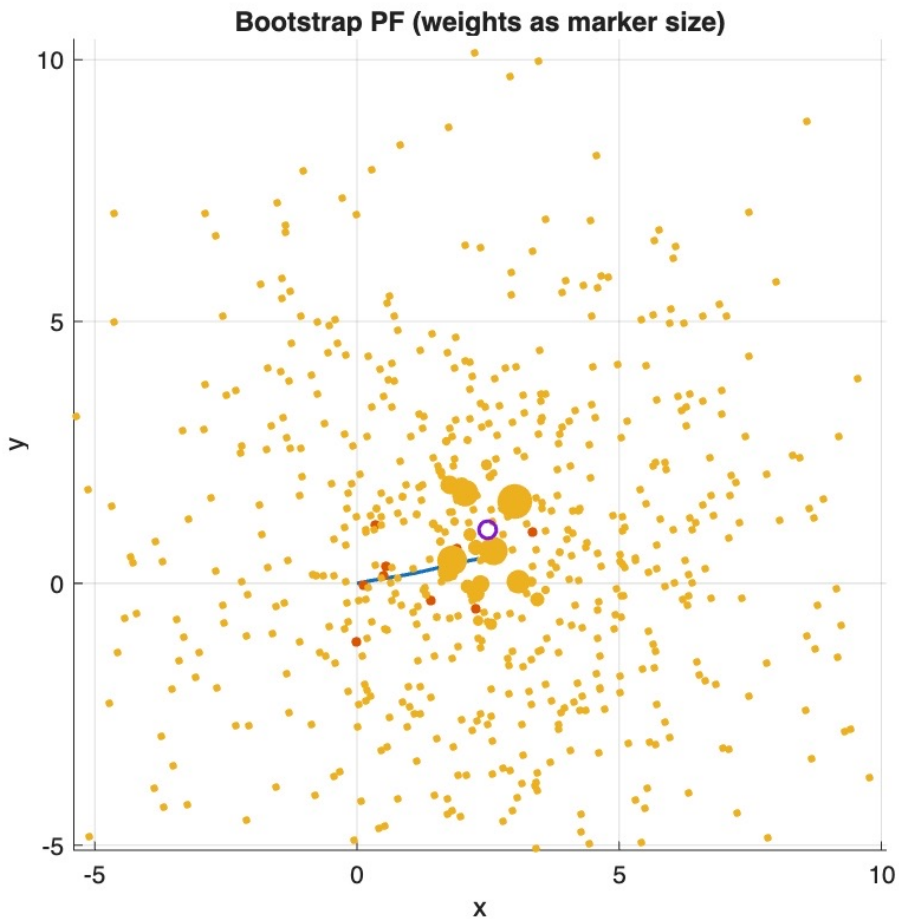
## Resampling Strategy

- There are multiple types of strategies which are used
- Most are stratified in some way
- This means that they guarantee that a range of points with very different weights gets selected
- This is to ensure that even low probability particles are maintained just in case they happen to be right
- One example is *systematic sampling*

## Systematic Sampling

- We'll explain systematic resampling with the aid of the linear system
- We are going to resample at time step 10
- This is actually a bit later than the ESS threshold would like us to, but it makes clearer what's going on


Bootstrap PF (weights as marker size)
k=10/1000
resampling=ON
ESS=4.9 (thr=330.0)
100
80
60
40
20
0
-20
-40
-60
-80
y
-80
-60
-40
-20
0
20
40
60
80
100
x

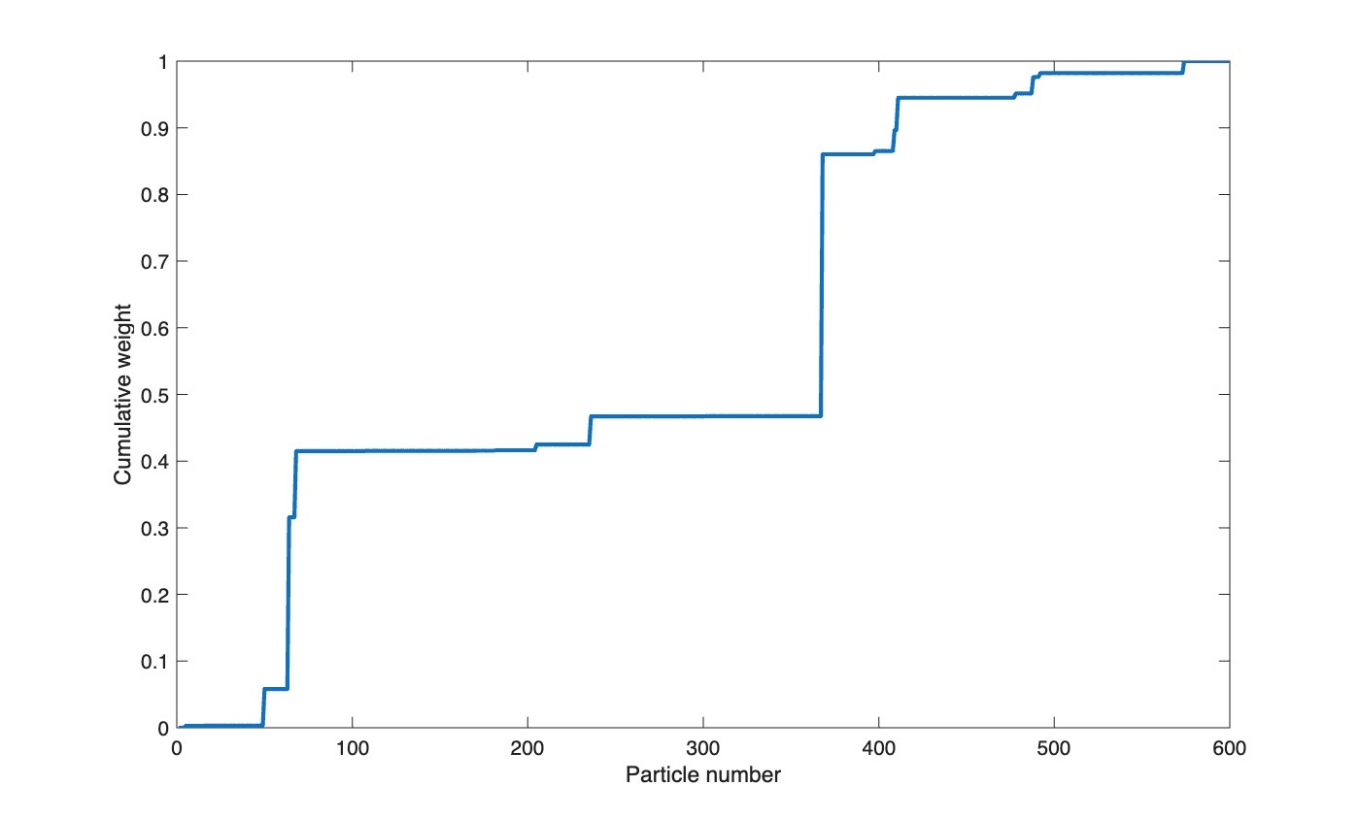

Bootstrap PF (weights as marker size)
10
5
0
-5
y
-5
0
5
10
x

## Systematic Sampling

- **We first compute the cumulative sum of the weights of all of the particles,**

$$\omega_{k|k}^{n} = \sum_{i=1}^{n} w_{k|k}^{[i]}$$

# Cumulative Sum



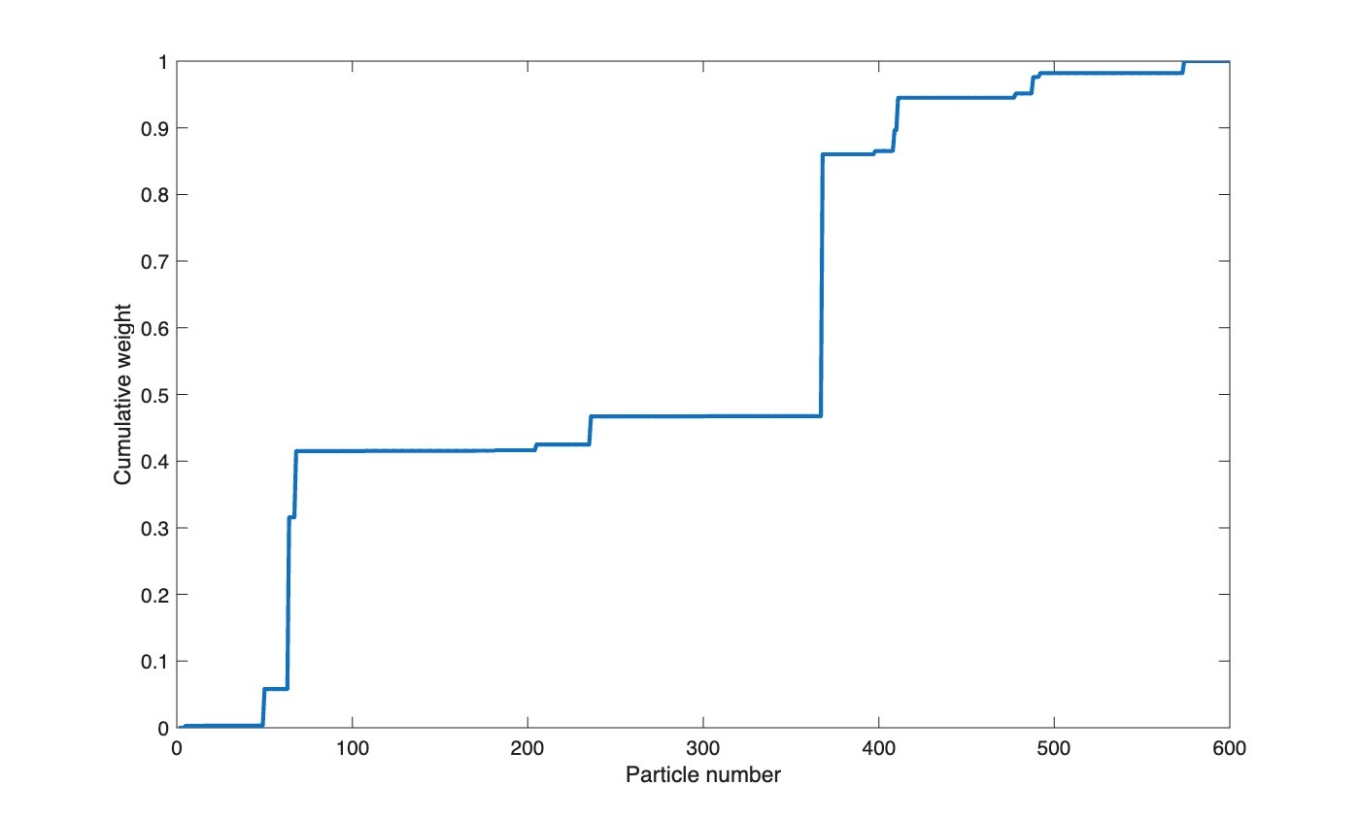
## Systematic Sampling

- We then draw a set of "selection probabilties",

$$U = \left\{ U^{[i]} \in [0,1] : i = 1, \ldots, N \right\}$$

- For each $U^i$ we go through the cumulative sum and find the index of the point where the cumulative sum first exceeds $U^i$

# Using $U^{[i]}$


Cumulative weight
Particle number
0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0.9
1
0
100
200
300
400
500
600

## Systematic Sampling

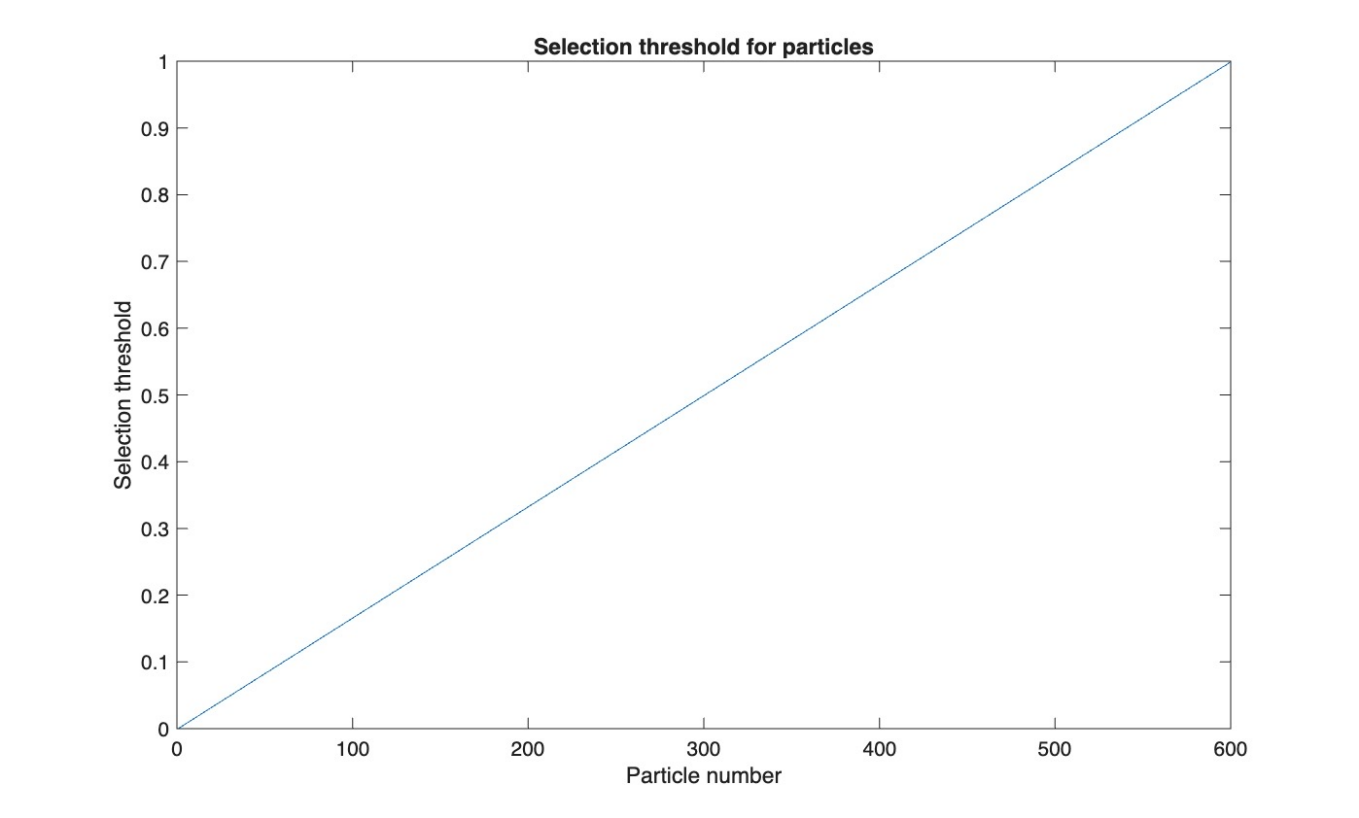
- Systematic sampling uses the following strategy to select $U^i$
- First sample an "offset"

$$U^{[0]} \sim \mathcal{U}\left(0, \frac{1}{N}\right)$$

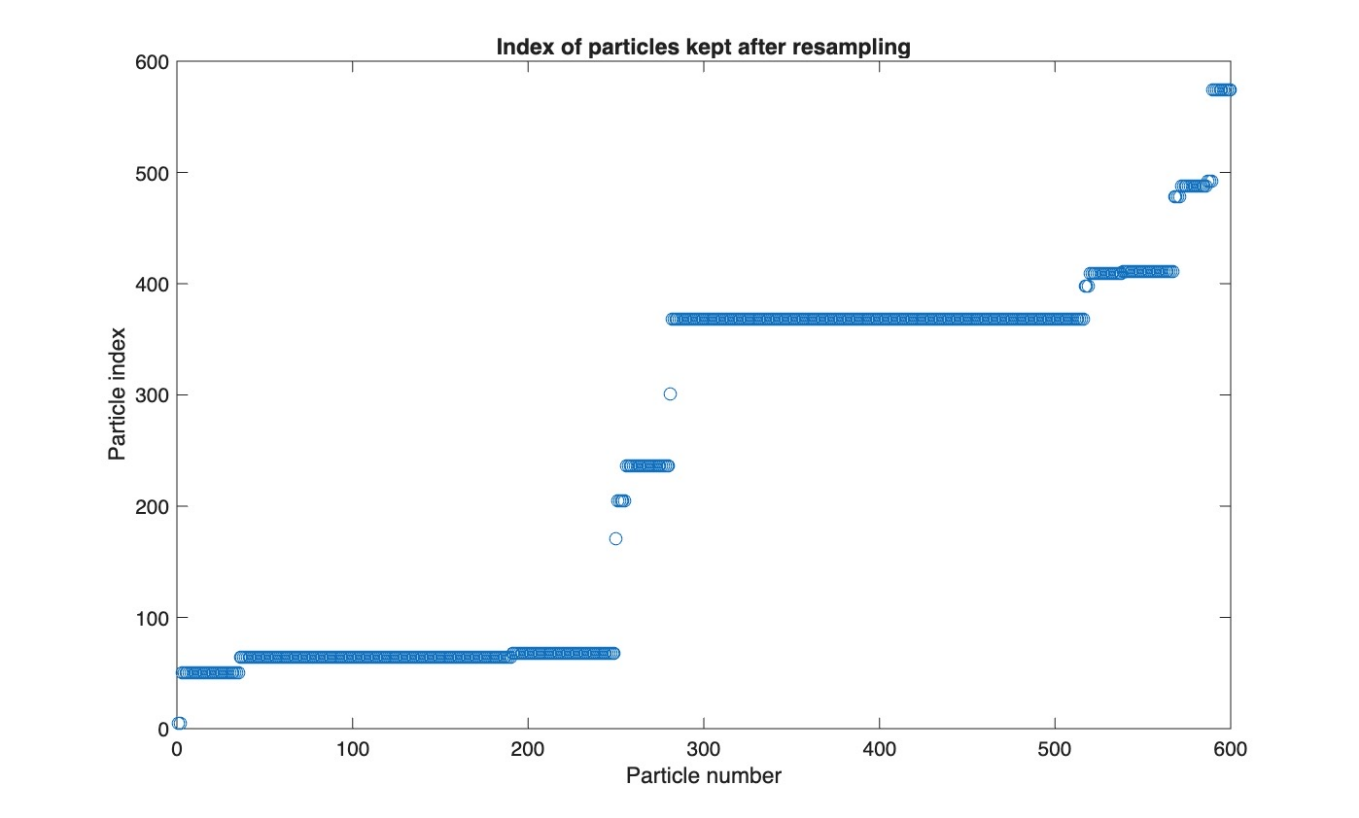
- Then select all the points according to

$$U^{[i]} = \frac{i-1}{N} + U^{[0]}$$

# Systematic Sampling $U^{[i]}$



# Systematic Sampling Resampled Particle Indices

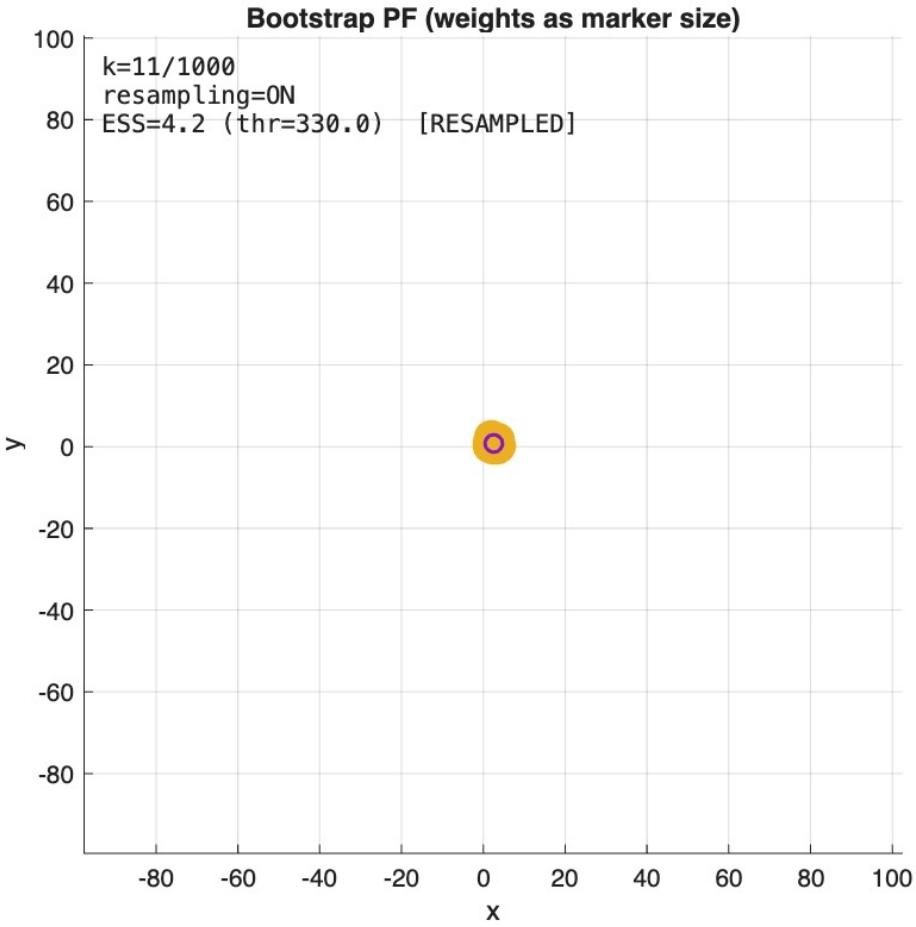


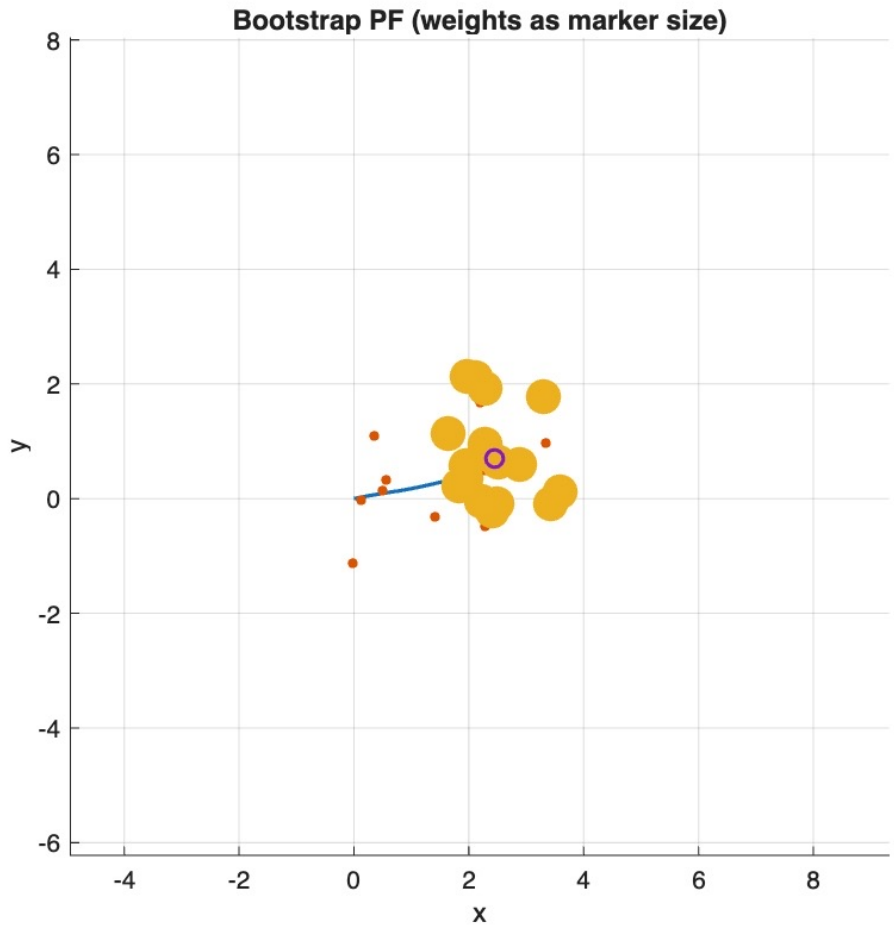
## Resampling

- Once the resampling step has been completed, all the particles are set to have the same weight,

$$w_{k|k}^{[i]} = \frac{1}{N}$$

- This also means that the ESS becomes $N$


Bootstrap PF (weights as marker size)
k=11/1000
resampling=ON
ESS=4.2 (thr=330.0)  [RESAMPLED]
100
80
60
40
20
0
-20
-40
-60
-80
y
-80
-60
-40
-20
0
20
40
60
80
100
x

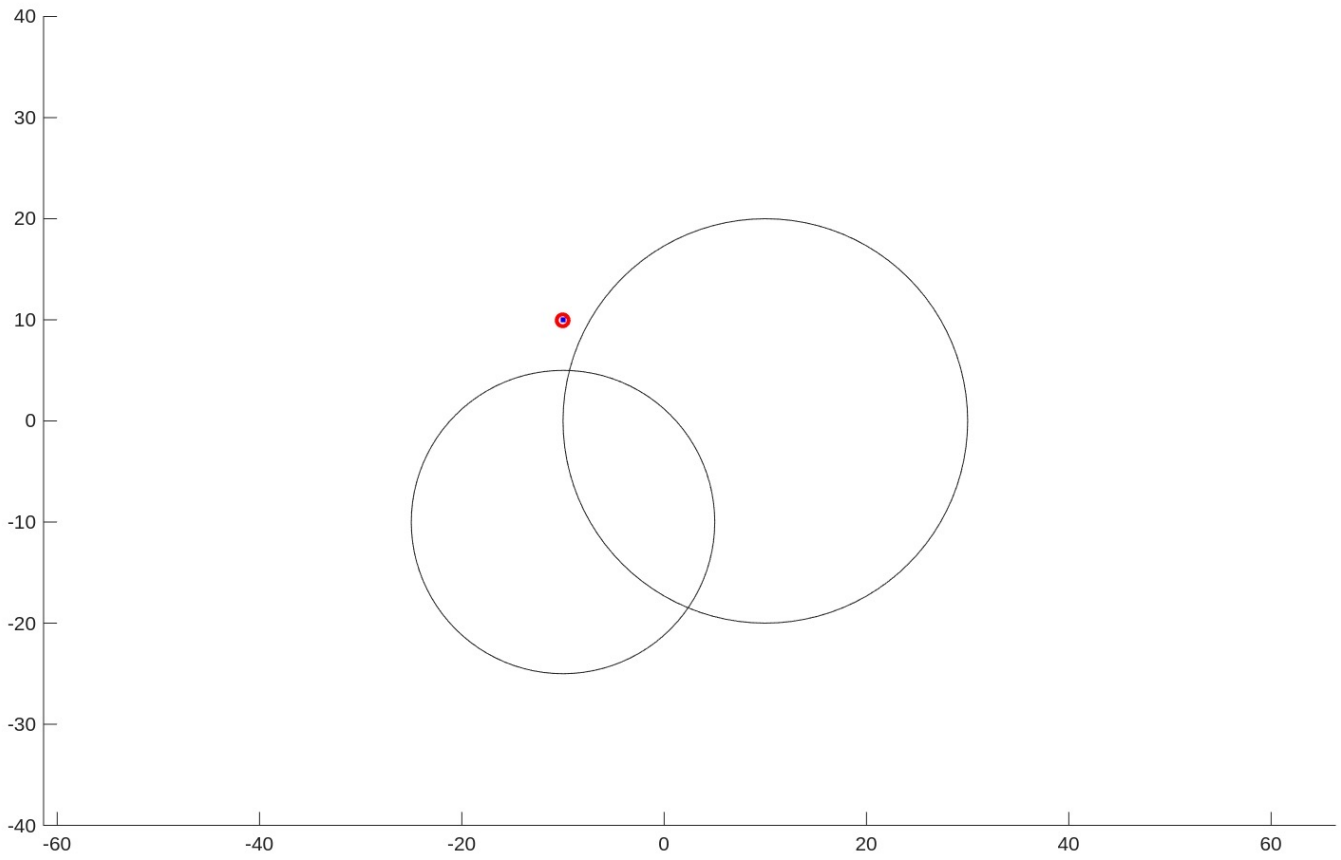

Bootstrap PF (weights as marker size)
8
6
4
2
0
-2
-4
-6
y
-4
-2
0
2
4
6
8
x

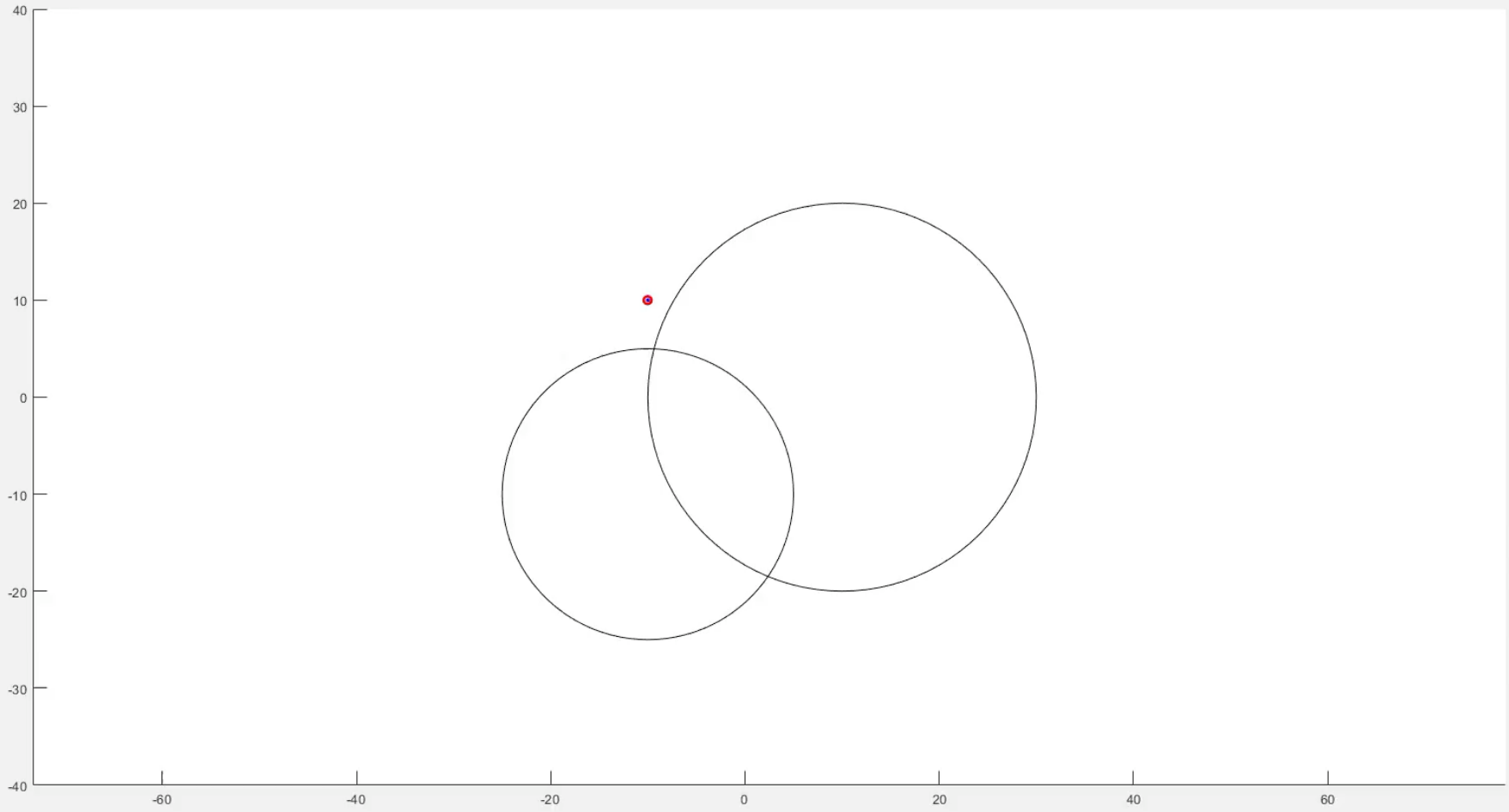
# A More Interesting Example

- *Problem Setup*
- Particle Approximation
- Sampling
- *Bootstrap Filter*
- *Effective Sample Size and Resampling*
- A More Interesting Example

# A More Interesting Example

- The goal of this example is to show a bit more the generality of the bootstrap filter by considering a highly nonlinear system
- The system consist of a 2D object whose position and velocity is to be estimated
- The object's position is detected by two range-only sensors


40
30
20
10
0
-10
-20
-30
-40
-60
-40
-20
0
20
40
60


40
30
20
10
0
-10
-20
-30
-40
-60
-40
-20
0
20
40
60

## Discussion

- Particle filters seem to be a very effective way to do estimation in nonlinear systems
- Their implementation is very straightforward, and they can model complex systems easily
- You might be thinking if that's the case, why don't we use these and ditch all these other approaches?
- However there are two issues: scale and accuracy

# Discussion

- **Scale:**
  - As the dimension increases, the number of particles has to increase dramatically
- **Accuracy:**
  - The random nature of the particles means that they fluctuate a lot
  - Therefore the estimates (e.g., mean) are rather “noisy” and tend to jump around a lot
- **Despite this, particle filters can be very useful**

## Where Next?

- In the next lecture, we'll look at occupancy grids
- These represent the world as a set of cells which might – or might not – be occupied
- Particle filters provide a principled way to handle platform uncertainty in these types of maps